

Documentación completa — Telecom X · Modelo de Evasión (Churn)

Material de estudio de nivel profesional. Este documento combina, en un solo texto, cinco formatos: un **libro universitario** (define cada concepto desde cero), un **manual de mejores prácticas** (qué hacer y qué evitar), una **guía de implementación real** (código y decisiones del proyecto), un **material de preparación para entrevistas técnicas** (intuición + matemática + casos límite) y los **apuntes de un buen profesor de Data Science** (analogías, ejemplos y advertencias).

Replica **todos** los bloques de `mejores_practicas.ipynb` y, sobre cada uno, añade: definición intuitiva y técnica de cada concepto, justificación estadística, intuición matemática, cuándo usar y cuándo NO, qué pasa si se omite, alternativas, errores típicos, reglas de decisión, ejemplos del mundo real y un resumen ejecutivo.

Lectura recomendada junto a `informe_auditoria.md` (diagnóstico del notebook original) y el notebook ejecutado.

Cómo usar este documento

Este material está pensado para **alguien que ya sabe de Data Science pero no necesariamente lo sabe todo**. No asumimos que conozcas de memoria qué es la *mutual information*, el *techo de Bayes* o por qué el *target encoding* puede arruinar un modelo. Cada vez que aparece un concepto técnico por primera vez, lo explicamos en profundidad mediante una **ficha de concepto**, para que la lectura sea fluida y no tengas que salir a buscar nada fuera.

Puedes leerlo de tres formas:

1. **De principio a fin**, como un curso: empieza por el *Framework general*, sigue por la *Filosofía* y recorre las 18 secciones del proyecto en orden. Es el camino recomendado la primera vez.
2. **Como manual de consulta**: ve directo a la sección que te interese; cada una es autocontenida y enlaza a las fichas de concepto relevantes.
3. **Como repaso rápido para una entrevista o una revisión**: lee solo los bloques **"Qué debes recordar"** al final de cada sección, las **Reglas de decisión** y la **Tabla maestra de referencia rápida** del final.

Convenciones (los recuadros que verás a lo largo del texto)

A lo largo del documento se repiten cinco tipos de bloque. Reconocerlos te ayuda a leer en diagonal cuando lo necesitas:

- **FICHA DE CONCEPTO — nombre**. Explicación profunda y autocontenida de un concepto técnico. Siempre responde, como mínimo: *qué es* (intuición + técnica), *para qué sirve*, *cuándo usarlo*, *cuándo NO usarlo*, *qué pasa si se omite* y una *analogía*. Es el corazón didáctico del documento.
- **INTUICIÓN MATEMÁTICA**. Cuando aparece una fórmula o una métrica, no nos limitamos a mostrarla: explicamos qué representa, cómo se lee, qué significa un valor alto y qué significa un valor bajo.
- **REGLA DE DECISIÓN**. Atajos del tipo "*si ocurre X → normalmente haz Y*". Son heurísticas, no dogmas: indican el camino por defecto y cuándo desviarse.
- **ERROR TÍPICO**. Un fallo frecuente en proyectos reales, su **consecuencia** concreta y cómo evitarlo.
- **QUÉ DEBES RECORDAR**. El resumen ejecutivo al final de cada sección: entre 3 y 10 ideas que deberían quedarte grabadas aunque olvides el resto.

Una nota sobre la honestidad de las cifras. Todos los números de este documento (F1, MCC, p-valores, gaps, intervalos de confianza) provienen de la ejecución real de `mejores_practicas.ipynb` con la semilla fijada. No son ilustrativos: son auditables y reproducibles. Cuando una cifra es aproximada o redondeada, se indica con " $\approx$ ".

Tabla de contenidos

Parte 0 — Marco conceptual

- [Cómo usar este documento](#)
- [Framework general para proyectos de clasificación](#)
- [Filosofía del proyecto](#)

Parte I — El proyecto, paso a paso (réplica del notebook)

- [1 · Configuración](#)
- [2 · Carga de datos](#)
- [3 · Auditoría de datos](#)
- [4 · EDA](#)
- [5 · Ingeniería de variables](#)
- [6 · Estrategia de validación](#)
- [7 · Preprocesamiento por evidencia](#)
- [8 · Baselines](#)
- [9 · Modelado](#)
- [10 · Estabilidad estadística](#)
- [11 · Optimización \(Optuna\)](#)
- [12 · Diagnóstico de generalización](#)
- [13 · Nested CV](#)
- [14 · Interpretabilidad](#)
- [15 · Selección final](#)
- [16 · Evaluación en test](#)
- [17 · Exportación](#)
- [18 · Conclusiones](#)

Parte II — Material transversal de referencia

- [19 · Reglas de decisión consolidadas](#)
- [20 · Catálogo de métricas: clasificación vs regresión](#)
- [21 · Tabla maestra de referencia rápida \(chuleta profesional\)](#)

Índice de fichas de concepto

Para ir directo a la explicación profunda de un concepto (se define en el lugar donde aparece por primera vez):

Concepto	Sección
Reproducibilidad y semilla aleatoria	1
Cardinalidad y variables identificador	2
Mutual Information (información mutua)	3
Multicolinealidad y VIF	3
Techo de Bayes (error irreducible)	3
Desbalance de clases	3
Correlación vs causalidad (confusión)	4
Trade-off sesgo–varianza	5 y 9
Data leakage (fuga de información)	6
Train/Test split y test aislado	6
Estratificación	6
K-Fold, Stratified K-Fold, Repeated SKF	6
Error estándar de la media (SEM) e IC	6 y 10
Escalado (Standard, Robust, MinMax, Power)	7
One-Hot, Ordinal y Target Encoding	7
Pipeline y ColumnTransformer	7
Baseline e hipótesis nula	8
Class weight / scale_pos_weight	9
Bagging vs Boosting	9
Overfitting, underfitting y gap train–CV	9
F1, Precision, Recall, Accuracy	9 y 15
MCC (coef. de Matthews)	9 y 15
ROC-AUC y PR-AUC	9 y 12
Test de Wilcoxon pareado	10
Comparaciones múltiples (Bonferroni/Holm)	10
Optimización bayesiana / TPE / Optuna	11
Pruning y early stopping	11
Regularización (C, L1/L2)	11
Curva de aprendizaje y de validación	12
Calibración de probabilidades	12
Nested Cross-Validation	13
SHAP (valores de Shapley)	14
Permutation importance	14
Coeficientes y log-odds	14
Optimización de umbral y F-beta	15
Navaja de Occam (parsimonia)	15
Training–serving skew y data drift	17

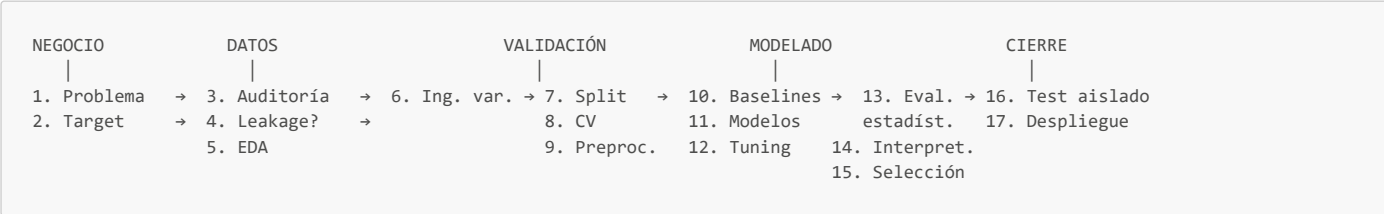
Framework general para proyectos de clasificación

Antes de entrar en el proyecto concreto, conviene tener en la cabeza el **mapa completo**. Un proyecto de Machine Learning serio no es "cargar datos y entrenar un modelo": es un proceso ordenado donde **cada paso protege al siguiente de un error específico**. Si te saltas un paso, el error que ese paso evitaba aparece más adelante disfrazado de "mala suerte" o de "el modelo no generaliza".

Esta es la metodología que sigue el proyecto Telecom X y que puedes reutilizar en casi cualquier problema de clasificación (y, con ajustes menores, de regresión). Piénsalo como una **lista de control mental**: en cada paso te haces las mismas preguntas y vigilas las mismas señales de alerta.

Idea rectora. El orden no es estético: es causal. El test se aísla *antes* de mirar nada (paso 7) porque cualquier decisión tomada después de "ver" el test lo contamina. El preprocesamiento se mete *dentro* de la validación (pasos 8–9) porque fuera de ella filtra información. La interpretabilidad va *después* de elegir el modelo (paso 14) porque explicar un modelo malo no sirve de nada. Cambiar el orden suele introducir el error que el orden evitaba.

Mapa de los 17 pasos



A continuación, cada paso con su **objetivo**, las **preguntas que responde**, los **errores frecuentes**, las **señales de alerta** y **qué métricas observar**.

Paso 1 · Comprender el problema de negocio

- **Objetivo.** Traducir una necesidad de negocio ("queremos retener clientes") a un problema de ML bien definido (¿clasificación binaria? ¿qué horizonte temporal? ¿qué se hace con la predicción?).
- **Preguntas que responde.** ¿Qué decisión va a cambiar el modelo? ¿Quién actúa sobre la predicción y con qué coste? ¿Cuánto cuesta un falso positivo (molestar/incentivar a un cliente fiel) frente a un falso negativo (perder a un cliente que se iba)? ¿Existe un modelo o regla actual que haya que superar?
- **Errores frecuentes.** Optimizar una métrica que al negocio no le importa; ignorar la **asimetría de costes** entre los dos tipos de error; construir un modelo que predice algo que llega demasiado tarde para actuar.
- **Señales de alerta.** Nadie sabe decir qué se hará con la predicción; "queremos predecir todo"; no hay un dueño de la decisión.
- **Qué observar.** No es una métrica todavía, sino un **marco de costes**: el coste relativo FN/FP guiará la métrica primaria y el umbral.

Paso 2 · Comprender la variable objetivo (target)

- **Objetivo.** Saber exactamente qué se predice y cómo está definida la etiqueta.
- **Preguntas que responde.** ¿Es binaria, multiclase, ordinal? ¿Cómo de **desbalanceada** está? ¿Cómo se construyó la etiqueta y en qué momento del tiempo (¿hay riesgo de que "el futuro" se haya colado en su definición)?
- **Errores frecuentes.** Aplicar codificaciones a la *target* (p. ej. One-Hot a la variable objetivo, que convierte el problema en otro distinto); definir la etiqueta usando información posterior a la predicción (leakage de definición).
- **Señales de alerta.** Clase positiva por debajo del ~10 %; etiquetas ambiguas; la definición del target menciona eventos futuros.
- **Qué observar.** Proporción de clases (`value_counts(normalize=True)`), tasa base de la clase positiva.

Paso 3 · Auditoría de calidad de datos

- **Objetivo.** Conocer el material con el que trabajas antes de modelar.
- **Preguntas que responde.** ¿Faltantes? ¿Duplicados? ¿Constantes o cuasi-constantas? ¿Tipos correctos? ¿Cardinalidad de cada variable? ¿Redundancias exactas entre columnas? ¿Multicolinealidad? ¿Cuánta señal tiene cada variable respecto al target (información mutua)? ¿Hay un **techo de Bayes** (perfiles idénticos con etiquetas opuestas)?
- **Errores frecuentes.** Saltarse esta fase y "descubrir" más tarde una columna identificador o una redundancia que distorsiona todo; eliminar variables solo por correlación baja (puede haber señal en interacción).
- **Señales de alerta.** Una variable con tantos valores únicos como filas (identificador); dos columnas con correlación ≈1; variables con un único valor.
- **Qué observar.** `%missing`, `nunique`, matriz de correlación, **VIF**, **mutual information** con el target.

Paso 4 · Identificación de data leakage

- **Objetivo.** Detectar, *antes* de modelar, cualquier vía por la que información del futuro o del test pueda colarse en el entrenamiento. Es el paso que más métricas infladas evita.
- **Preguntas que responde.** ¿Hay variables que solo existen *después* de que ocurra el evento (p. ej. "motivo de baja")? ¿El preprocesamiento aprende parámetros usando datos de test? ¿Se eligen features o umbrales mirando el test?
- **Errores frecuentes.** Escalar/imputar/codificar sobre todo el dataset antes de separar train/test; seleccionar variables evaluando en el test; *target encoding* con la media global.
- **Señales de alerta.** Una variable "demasiado buena" (predice casi perfecto); métricas de test mejores que las de validación; una columna cuyo significado depende de que el evento ya haya ocurrido.
- **Qué observar.** Coherencia entre métrica de CV y de test (si test ≫ CV, sospecha leakage); importancia desproporcionada de una sola variable.

Paso 5 · Análisis exploratorio (EDA)

- **Objetivo.** Formar hipótesis sobre qué variables discriminan y cómo, y entender la forma de los datos.
- **Preguntas que responde.** ¿Qué categorías tienen tasas de evento muy distintas? ¿Hay relaciones no lineales? ¿Distribuciones sesgadas, outliers, interacciones evidentes?

- **Errores frecuentes.** Confundir **correlación con causalidad**; sacar conclusiones de un análisis solo univariado; "decorar" con gráficos sin extraer una hipótesis accionable.
- **Señales de alerta.** Diferencias entre grupos que desaparecen al controlar por una tercera variable (confusión).
- **Qué observar.** Tasas condicionales $P(y \mid \text{categoría})$, distribuciones por clase, correlaciones, tablas de contingencia.

Paso 6 · Ingeniería de variables

- **Objetivo.** Crear, transformar o eliminar variables para ayudar al modelo, **sin** introducir leakage ni complejidad innecesaria.
- **Preguntas que responde.** ¿Qué variable es redundante y se puede eliminar? ¿Qué transformación tiene sentido de dominio? ¿Una feature nueva mejora la CV o solo el train?
- **Errores frecuentes.** Ingeniería agresiva que mejora train y empeora test; crear features con estadísticas globales (leakage); añadir variables "por si acaso".
- **Señales de alerta.** El train sube y la CV no; el número de features crece sin justificación por validación.
- **Qué observar.** Cambio en F1/MCC de **CV** (no de train) al añadir/quitar una feature.

Paso 7 · Separación Train/Test (aislar el test)

- **Objetivo.** Apartar una muestra que **no se vuelve a tocar** hasta el final, para tener una estimación honesta de generalización.
- **Preguntas que responde.** ¿Qué tamaño de test da suficiente precisión? ¿Estratificar por la clase? ¿Hay estructura temporal o de grupos que obligue a un split especial?
- **Errores frecuentes.** Mirar el test "solo para echar un vistazo" (lo quemamos); split no estratificado con clases desbalanceadas; fuga de grupos (el mismo cliente en train y test).
- **Señales de alerta.** Cualquier decisión tomada después de ver el test; proporción de clases distinta entre train y test.
- **Qué observar.** Que la proporción de la clase positiva sea ~igual en train y test; tamaño absoluto de la clase minoritaria en test.

Paso 8 · Validación cruzada (diseñar el protocolo)

- **Objetivo.** Definir cómo se estimará el rendimiento *dentro* de train para tomar todas las decisiones.
- **Preguntas que responde.** ¿K-Fold, Stratified, Repeated, Nested? ¿Cuántos folds y repeticiones? ¿Cuánta incertidumbre (SEM) puedo permitirme?
- **Errores frecuentes.** Hold-out único en dataset pequeño (estimación ruidosa); no estratificar con clases desbalanceadas; usar la misma CV para tunear y reportar (sesgo optimista → hace falta nested).
- **Señales de alerta.** Desviación entre folds muy alta; un fold con muy pocos positivos.
- **Qué observar.** Media y **desviación entre folds**, $SEM = s/\sqrt{k}$, intervalos de confianza.

Paso 9 · Preprocesamiento (dentro de pipelines)

- **Objetivo.** Escalar, codificar e imputar **dentro** de un **Pipeline**, de modo que cada fold ajuste el preprocesador solo con su train.
- **Preguntas que responde.** ¿Qué escalador y qué codificador, decididos por evidencia (CV) y no por costumbre? ¿Qué modelos necesitan escalado y cuáles no?
- **Errores frecuentes.** `fit_transform` sobre todo el dataset (leakage L1); escalar árboles (esfuerzo inútil); *target encoding* sin CV interna.
- **Señales de alerta.** El preprocesador "vive" fuera del flujo de validación; el scaler se ajustó una sola vez.
- **Qué observar.** F1 de CV comparada entre opciones de preprocesamiento (no de train).

Paso 10 · Baselines

- **Objetivo.** Fijar un **piso** cuantitativo que todo modelo serio debe superar para justificar su coste.
- **Preguntas que responde.** ¿Cuánto rinde "predecir siempre la mayoría"? ¿Y al azar respetando proporciones? ¿Y un modelo lineal simple?
- **Errores frecuentes.** No tener baseline y celebrar una métrica que un **DummyClassifier** igualaría; usar Accuracy como referencia en problemas desbalanceados.
- **Señales de alerta.** Accuracy alta pero Recall/F1 de la clase positiva ≈ 0 .
- **Qué observar.** Accuracy del Dummy (para desenmascararla), F1/MCC/PR-AUC de la clase positiva.

Paso 11 · Comparación de modelos (zoo de modelos)

- **Objetivo.** Probar familias diversas (lineal, kernel, instancias, bagging, boosting) con el mismo protocolo, para tener evidencia de si la complejidad **realmente** aporta.
- **Preguntas que responde.** ¿Qué familia rinde mejor en CV? ¿Cuánto sobreajusta cada una (gap train-CV)?
- **Errores frecuentes.** Probar un solo modelo "de moda"; comparar con umbral 0.5 sin mirar métricas independientes del umbral; declarar ganador por milésimas.
- **Señales de alerta.** Un modelo con $F1_{\text{train}} \approx 1.0$ y $F1_{\text{CV}}$ mediocre (memoriza).
- **Qué observar.** F1, MCC, PR-AUC, ROC-AUC y, sobre todo, el **gap train-CV**.

Paso 12 · Optimización de hiperparámetros (tuning)

- **Objetivo.** Afinar los modelos prometedores **sin** mirar el test y **sin** sobreajustar al propio proceso de tuning.
- **Preguntas que responde.** ¿Qué hiperparámetros importan y en qué rango? ¿El tuning cambia el veredicto entre modelos?
- **Errores frecuentes.** Tunear contra el test; espacios de búsqueda mal acotados; `eval_set` de early stopping mal transformado (reintroduce leakage); creer que más trials siempre es mejor.
- **Señales de alerta.** El gap train-CV crece tras tunear; mejoras que no sobreviven a la nested CV.
- **Qué observar.** F1 de CV antes/después, gap train-CV del modelo tuneado.

Paso 13 · Evaluación estadística (¿la diferencia es real?)

- **Objetivo.** Decidir si una diferencia entre modelos es **señal o ruido** de muestreo.
- **Preguntas que responde.** ¿El mejor modelo es significativamente mejor que los demás, o es un empate técnico?
- **Errores frecuentes.** Declarar ganador por 0.001 de F1; ignorar las **comparaciones múltiples**; usar un t-test cuando las diferencias no son normales.

- **Señales de alerta.** Intervalos de confianza que se solapan ampliamente; p-valores > 0.05.
- **Qué observar.** IC 95 % de cada modelo, **p-valor de un test pareado** (Wilcoxon) entre el mejor y cada rival.

Paso 14 · Interpretabilidad

- **Objetivo.** Entender *por qué* predice el modelo, no solo *qué* predice; validar que usa señales sensatas.
- **Preguntas que responde.** ¿Qué variables empujan la predicción y en qué dirección? ¿Coinciden distintos métodos? ¿Se puede explicar un caso individual?
- **Errores frecuentes.** Confundir importancia con causalidad; fiarse solo de `feature_importances_` (sesgada hacia alta cardinalidad); ignorar la multicolinealidad al leer importancias.
- **Señales de alerta.** El modelo se apoya en una variable que no debería tener señal; métodos de interpretación que se contradicen.
- **Qué observar.** Coeficientes/log-odds, **permutation importance**, **SHAP**; convergencia entre métodos.

Paso 15 · Selección final del modelo

- **Objetivo.** Elegir, entre los candidatos estadísticamente equivalentes, el mejor según la jerarquía del proyecto.
- **Preguntas que responde.** A igualdad de rendimiento, ¿cuál es más simple, estable, interpretable y barato? ¿Qué **umbral** de decisión conviene?
- **Errores frecuentes.** Elegir el más complejo "por si acaso"; optimizar el umbral mirando el test; quedarse en el 0.5 por defecto con clases desbalanceadas.
- **Señales de alerta.** El modelo elegido solo gana por milésimas y a costa de mucha varianza.
- **Qué observar.** Estabilidad (gap), interpretabilidad, coste de inferencia; F1/MCC al umbral óptimo *out-of-fold*.

Paso 16 · Evaluación final en el test aislado

- **Objetivo.** Medir, **una sola vez**, el rendimiento honesto de generalización del pipeline ganador.
- **Preguntas que responde.** ¿La métrica de test coincide con la de CV/nested CV? (Si sí → sin leakage y generaliza.)
- **Errores frecuentes.** Ajustar algo después de ver el test; reportar una sola cifra sin intervalo.
- **Señales de alerta.** Test claramente peor que CV (leakage en el protocolo) o claramente mejor (test contaminado).
- **Qué observar.** Todas las métricas en test, comparadas con CV; idealmente un IC por bootstrap del test.

Paso 17 · Despliegue (MLOps)

- **Objetivo.** Llevar el modelo a producción de forma reproducible, sin divergencia entre entrenamiento e inferencia.
- **Preguntas que responde.** ¿Cómo se empaqueta (pipeline + umbral + metadatos)? ¿Cómo se versiona? ¿Cómo se monitoriza el *drift*?
- **Errores frecuentes.** Reimplementar el preprocesamiento a mano en producción (*training-serving skew*); no guardar la versión de las librerías; no monitorizar el cambio de población.
- **Señales de alerta.** El preprocesamiento de inferencia difiere del de entrenamiento; degradación silenciosa de métricas en producción.
- **Qué observar.** En producción: estabilidad de la distribución de entrada (*data drift*), métricas sobre etiquetas reales cuando lleguen, latencia de inferencia.

☑ QUÉ DEBES RECORDAR (del framework)

1. El orden de los pasos es **causal**: cada uno evita un error concreto del siguiente.
2. El test se **aisla pronto** (paso 7) y se **toca una sola vez** (paso 16). Todo lo demás vive en train + CV.
3. El preprocesamiento va **dentro** de la validación (pipelines), nunca antes del split.
4. Antes de modelar: **auditoría + caza de leakage**. Es donde se ganan o se pierden las métricas honestas.
5. **Baselines primero**, comparación con incertidumbre después, e interpretabilidad solo del modelo que vas a usar.
6. La elección final no es "el F1 más alto", sino "el mejor entre los estadísticamente equivalentes" según una jerarquía explícita (generalización > leakage > robustez > reproducibilidad > interpretabilidad > poder predictivo).

Filosofía del proyecto

El objetivo **no** es maximizar una métrica en una partición concreta, sino **estimar y maximizar el rendimiento sobre datos nunca vistos**. Esta frase parece obvia, pero casi todos los errores graves de un proyecto de ML vienen de olvidarla: se optimiza el número que se ve (la métrica en *estos* datos) en lugar del número que importa (la métrica en los datos que *aún no existen*). Para no caer en esa trampa, el proyecto subordina todo a una **jerarquía explícita**:

1. **Generalización real** → medida con protocolos que no se "auto-engañan" (CV repetida, nested CV, test aislado).
2. **Ausencia de data leakage** → el modelo nunca ve, ni indirectamente, información del futuro o del test.
3. **Robustez estadística** → toda diferencia entre modelos se contrasta contra el ruido de muestreo.
4. **Reproducibilidad** → semilla fija, pipelines cerrados, artefactos versionados.
5. **Interpretabilidad** → entender *por qué* predice, no solo *qué* predice.
6. **Poder predictivo** → último en la lista. No porque no importe, sino porque **sin los cinco anteriores, una métrica alta es una ilusión**: mide el pasado, no el futuro.

Por qué el poder predictivo va al final (y no es una contradicción). A primera vista suena raro poner "predecir bien" en el último lugar de un proyecto cuyo fin es predecir. La clave es distinguir entre la métrica *medida* y la métrica *real*. Un modelo con leakage puede mostrar $F1 = 0.95$ en tu pantalla y rendir 0.60 en producción: el 0.95 es poder predictivo aparente, el 0.60 es el real. La jerarquía dice: "asegúrate primero de que el número que ves es el número que obtendrás; solo entonces tiene sentido maximizarlo". Es la diferencia entre una **métrica de marketing** y una **métrica de ingeniería**.

📄 FICHA DE CONCEPTO — Data leakage (fuga de información) (se profundiza en la sección 6; aquí va la intuición)

- **Qué es.** Cualquier situación en la que información que **no estará disponible en el momento real de predecir** (o que pertenece al conjunto de test/futuro) se filtra en el entrenamiento o en las decisiones de modelado. El modelo "aprende" usando pistas que no tendrá cuando trabaje de verdad.
- **Por qué es tan grave.** Infla las métricas durante el desarrollo y las desploma en producción. Es traicionero porque **no se ve en la lista de columnas**: a menudo es de *proceso* (escalar antes de separar, elegir features mirando el test), no de contenido.

- **Ejemplo intuitivo del leakage de proceso.** Imagina que estudias para un examen y, sin darte cuenta, "calibras" tu método de estudio mirando las respuestas del examen real. Sacarás un 10 en ese examen y un 5 en el siguiente. El leakage hace exactamente eso: ajusta decisiones usando datos que deberían estar sellados, y la nota (métrica) deja de predecir el rendimiento futuro.
- **Cómo se combate en este proyecto.** Aislado el test desde el principio, metiendo todo el preprocesamiento dentro de pipelines (que se reajustan en cada fold) y tomando todas las decisiones por validación cruzada.

Este documento es, en el fondo, la historia de cómo esa jerarquía lleva, paso a paso y con evidencia, a elegir un modelo **simple** (Regresión Logística) sobre alternativas más sofisticadas que no generalizan mejor. El notebook original (auditado en [informe_auditoria.md](#)) hacía lo contrario: maximizó recall sobre datos contaminados por leakage y eligió un Random Forest cuyo rendimiento reportado estaba inflado. La reconstrucción corrige cada uno de esos puntos.

☑ QUÉ DEBES RECORDAR (filosofía)

1. El fin no es la métrica que ves, sino la que obtendrás en datos nuevos.
2. Una métrica alta solo vale si está libre de leakage, es estadísticamente robusta y reproducible.
3. A igualdad de rendimiento, gana lo **simple, estable e interpretable** (lo justificaremos formalmente con la navaja de Occam en la sección 15).
4. El poder predictivo es el último eslabón porque depende de que los anteriores se cumplan.

1 · Configuración

```
import warnings, time, json
warnings.filterwarnings("ignore")
import numpy as np, pandas as pd
import matplotlib.pyplot as plt, seaborn as sns
from scipy import stats
# ... sklearn, xgboost, lightgbm, catboost, optuna, shap ...
RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)
optuna.logging.set_verbosity(optuna.logging.WARNING)
```

Qué se hizo. Centralizamos los imports y, sobre todo, fijamos `RANDOM_STATE = 42` y lo propagamos a *cada* fuente de azar (particiones de datos, inicialización y submuestreo de los modelos, sampler de Optuna). Definimos además una única función de métricas ([metricas_completas](#)) para que **todas las fases midan exactamente lo mismo**, evitando comparar peras con manzanas entre secciones.

Por qué se hizo (justificación). La aleatoriedad entra en un proyecto de ML por tres puertas distintas:

1. **La partición de los datos** (qué filas caen en train, en test y en cada fold).
2. **El propio modelo** (inicialización de pesos, submuestreo de filas/columnas en Random Forest, orden de los árboles...).
3. **El optimizador de hiperparámetros** (Optuna muestrea configuraciones de forma estocástica).

Si no fijas la semilla, dos ejecuciones del mismo código dan resultados distintos, y cualquier comparación entre modelos queda contaminada por **varianza que no controlas**: no sabrías si "el modelo A es mejor que B" o si "esta vez A tuvo suerte con la partición". Fijar la semilla convierte el experimento en **determinista y auditable**: cualquiera que ejecute el notebook obtiene exactamente tus números.

📄 FICHA DE CONCEPTO — Reproducibilidad y semilla aleatoria (random seed)

- **Qué es.** Una semilla es el número que inicializa el generador de números pseudoaleatorios. Fijarla hace que la secuencia "aleatoria" sea siempre la misma, y por tanto que todo lo que depende del azar (splits, modelos, tuning) se repita idéntico.
- **Para qué sirve.** Para que los resultados sean **reproducibles** (otra persona, u otro día, obtiene lo mismo) y para que las comparaciones entre modelos sean **limpias** (todos ven exactamente las mismas particiones).
- **Cuándo usarla.** Siempre, en investigación y en producción. Es gratis y es la base de la auditoría.
- **Cuándo NO basta.** Cuando quieres saber si tu resultado depende de *una* semilla concreta. Entonces conviene repetir con varias semillas y promediar, o usar validación repetida (que es justo lo que hace la RSKF de la sección 6).
- **Qué pasa si se omite.** Resultados irreproducibles, comparaciones no concluyentes, y la imposibilidad de distinguir una mejora real de una casualidad.
- **Analogía.** Es como repartir las cartas siempre desde una baraja barajada de la misma forma: todos los jugadores (modelos) reciben exactamente la misma mano, así que si uno gana es por cómo juega, no por las cartas que le tocaron.

📖 **INTUICIÓN MATEMÁTICA — el "42" no es mágico.** El valor concreto de la semilla es arbitrario; lo importante es que sea **fijo**. Que el resultado dependa fuertemente de elegir 42 frente a 7 sería en sí una señal de alarma (alta varianza). Por eso el proyecto no se fía de una sola semilla: usa validación **repetida** (15 particiones) y nested CV, de modo que el veredicto no cuelga de un número concreto.

⚠ **ERROR TÍPICO — sobreajustar a la semilla.** *Error:* presentar un resultado que solo se sostiene con `random_state=42` y se cae con otra semilla. *Consecuencia:* das por buena una mejora que es ruido de partición; en producción (otra "semilla" de facto) desaparece. *Cómo evitarlo:* validación repetida y reportar la **variabilidad** entre folds, no solo la media.

⚠ **ERROR TÍPICO — silenciar todos los warnings.** *Error:* `warnings.filterwarnings("ignore")` de forma permanente. *Consecuencia:* ocultas avisos útiles (no-convergencia, valores deprecados, división por cero). Aquí se asume porque el ruido de convergencia de algunos modelos satura la salida, pero **en depuración conviene reactivarlos**.

Alternativas. `sklearn.utils.check_random_state` para propagar la semilla de forma idiomática; ejecutar todo bajo varias semillas y promediar (más robusto, más caro). Para reproducibilidad *bit a bit* habría que fijar también `PYTHONHASHSEED` y el número de hilos de BLAS, porque las sumas en coma flotante no son asociativas y el paralelismo puede alterar el último dígito.

☑ QUÉ DEBES RECORDAR (configuración)

1. Hay **tres fuentes de azar**: partición, modelo y optimizador. Fíjalas todas con una única semilla global.

2. La semilla da **reproducibilidad y comparaciones limpias**; no mejora el modelo, solo lo hace honesto.

3. No te fíes de **una** semilla: válida de forma repetida para no confundir suerte con señal.

4. Silenciar warnings es cómodo pero peligroso; reactivalos al depurar.

2 · Carga de datos

```
df = pd.read_csv("data_limpia.csv")
df = df.drop(columns=["customerid"]) # ID único: 0 señal
```

Qué se hizo. Cargamos el **mismo** archivo que el proyecto original (copiado sin modificar, 7032 filas × 22 columnas) y eliminamos `customerid`. Esa columna tiene 7032 valores únicos en 7032 filas: es una **clave primaria**, no una variable predictora.

Por qué se hizo. Un identificador con tantos valores distintos como filas es el caso extremo de **variable de alta cardinalidad**, y es predictivamente inútil (y peligroso). Dentro de train, un modelo flexible podría "memorizar" el mapa `id → churn` y acertar el 100 %... para luego no saber nada de un cliente nuevo, cuyo id jamás ha visto.

FICHA DE CONCEPTO — Cardinalidad y variables identificador

- **Qué es la cardinalidad.** El número de valores distintos que toma una variable. `gender` tiene cardinalidad 2; `account_contract` tiene 3; `customerid` tiene 7032 (= nº de filas).
- **Por qué un identificador es venenoso como feature.** Tiene **información mutua máxima con el target dentro de train** (cada id se asocia a un único desenlace) pero **cero capacidad de generalizar** (los ids nuevos no aparecen en train). Es la receta perfecta del sobreajuste: el modelo memoriza en lugar de aprender un patrón.
- **Cuándo conviene conservar un ID o derivar algo de él.** Solo si codifica información real: por ejemplo, si el id es creciente con la fecha de alta, podrías extraer "antigüedad de registro" como feature. Aquí no es el caso.
- **Cuándo eliminarlo.** Siempre que sea un identificador arbitrario sin orden ni significado. Como aquí.
- **Qué pasa si se omite la eliminación.** Leakage trivial pero devastador: árboles y modelos de alta capacidad lo usan para memorizar, y el rendimiento reportado se vuelve fantasía.
- **Analogía.** Predecir si un alumno aprueba usando su número de matrícula: en el grupo conocido "aciertas" memorizando, pero con alumnos nuevos no sabes absolutamente nada.

REGLA DE DECISIÓN — variables de muy alta cardinalidad

- Si una columna tiene **tantos valores únicos como filas** (o casi) → es un identificador → **elimínala** (salvo que codifique orden/tiempo útil, en cuyo caso extrae esa señal).
- Si una categórica tiene **cardinalidad alta pero finita** (decenas/cientos: códigos postales, ciudades) → no la tires; evalúa **target/frequency encoding** (con CV interna) en lugar de One-Hot. (Ver sección 7.)

ERROR TÍPICO — dejar el ID dentro de X. *Error:* no eliminar `customerid` (o un email, DNI, número de pedido...). *Consecuencia:* sobreajuste perfecto en train, métricas infladas y un modelo inútil con clientes nuevos.

Impacto esperado. Eliminamos una fuente de sobreajuste y reducimos dimensionalidad inútil sin perder absolutamente nada de señal.

☑ QUÉ DEBES RECORDAR (carga)

1. Cardinalidad = número de valores distintos; vigílala siempre en la auditoría.

2. Un identificador único es **señal cero y riesgo máximo**: elimínalo.

3. Alta cardinalidad *finita* no se tira: se codifica con cuidado (target/frequency con CV).

3 · Auditoría de datos (FASE 2)

```
audit = pd.DataFrame({"dtype": df.dtypes, "n_unique": df.nunique(),
                      "%_missing": df.isna().mean()*100})
# redundancia determinística
np.allclose(df["account_charges_day"], df["account_charges_monthly"]/30) # True
# multicolinealidad
df["account_charges_total"].corr(df["account_charges_monthly"]*df["customer_tenure"]) # 0.9996
# mutual information con churn (mutual_info_classif)
# perfiles idénticos con etiquetas opuestas -> techo de Bayes
```

Qué se hizo. Una auditoría sistemática del dataset *antes* de modelar: tipos, faltantes, duplicados, constantes, cardinalidad, redundancias exactas, multicolinealidad, información mutua con el target y existencia de un techo de Bayes. La idea es **conocer el material** con el que vamos a trabajar para no llevarnos sorpresas más adelante.

Hallazgos (todos verificados sobre el dataset).

Hallazgo	Evidencia	Acción
Sin valores faltantes	<code>%_missing = 0</code> en las 22 columnas	Ninguna imputación necesaria
Sin duplicados exactos	<code>df.duplicated().sum() == 0</code>	—

Hallazgo	Evidencia	Acción
Sin constantes ni cuasi-constantes	clase minoritaria mínima ≈ 9.7 % (<code>phone_phoneservice</code>)	Se conservan
<code>charges_day = monthly/30</code>	ratio constante = 30.0	Eliminar (redundante exacta)
<code>charges_total ≈ monthly×tenure</code>	corr = 0.9996	Vigilar multicolinealidad
MI más alta	<code>account_contract</code> (0.098), <code>tenure</code> (0.074), <code>internet_service</code> (0.055)	Predictores núcleo
MI ≈ 0	<code>gender</code> (0.000), <code>phone_phoneservice</code> (0.0001), <code>streaming</code> (~0.002)	Ruido casi puro
18 perfiles con churn en conflicto	mismas features, distinto resultado	Techo de Bayes (error irreducible)
Desbalance	churn = 26.6 %	Tratar con pesos de clase

Vamos a desmenuzar los cuatro conceptos no triviales de esta tabla: **información mutua**, **multicolinealidad/VIF**, **techo de Bayes** y **desbalance**.

FICHA DE CONCEPTO — Mutual Information (información mutua, MI)

- **Qué es (intuitivo).** Cuánto se *reduce tu incertidumbre* sobre el target al conocer el valor de una variable. Si saber el tipo de contrato te ayuda mucho a adivinar si el cliente se irá, la MI entre `contrato` y `churn` es alta. Si conocer el `gender` no cambia nada tu apuesta, su MI es ≈ 0.
- **Qué es (técnico).** Mide, en *nats* (o bits), la información compartida entre dos variables: $MI(X,Y) = \sum p(x,y) \cdot \log[p(x,y) / (p(x) \cdot p(y))]$. Es 0 si y solo si X e Y son **independientes**, y crece cuanto más se aleja la distribución conjunta del producto de las marginales.
- **Para qué sirve.** Para clasificar variables por su señal bruta respecto al target, **incluyendo relaciones no lineales y no monótonas**, y funcionando tanto con numéricas como con categóricas.
- **En qué se diferencia de la correlación de Pearson.** Pearson solo capta relaciones **lineales** y solo entre variables numéricas; una relación en forma de U tiene correlación ≈ 0 pero MI alta. La MI ve patrones que Pearson no ve.
- **Cuándo usarla.** En la auditoría, para tener un *ranking* rápido de relevancia y detectar variables que son ruido casi puro. En selección de variables (`mutual_info_classif`/`mutual_info_regression`).
- **Cuándo NO fiarse de ella en solitario.** La MI es **univariada**: mide cada variable *aisladamente*. Una variable con MI ≈ 0 puede ser muy útil **en interacción** con otra (XOR es el ejemplo de libro: cada variable sola es inútil, juntas lo explican todo). Por eso **no eliminamos variables solo por MI baja**.
- **Qué pasa si se omite.** Modelas a ciegas: no sabes qué variables son núcleo y cuáles ruido, ni dónde concentrar la interpretación.
- **Analogía.** La MI es como preguntar "¿cuánto me sirve esta pista para resolver el caso?". Una huella dactilar (`contrato`) reduce mucho los sospechosos; el color de los calcetines (`gender`) no reduce nada.

INTUICIÓN MATEMÁTICA — leer los valores de MI. En este dataset, `account_contract` tiene MI ≈ 0.098 y `gender` ≈ 0.000. Los valores absolutos de MI no tienen una escala "universal" fácil de interpretar (dependen del número de categorías y del tamaño de muestra), así que se leen **en términos relativos**: `contrato` aporta ~100× más información que `gender`. Un valor ≈ 0 es interpretable de forma tajante: **independencia** → esa variable, por sí sola, no dice nada del churn.

FICHA DE CONCEPTO — Multicolinealidad y VIF

- **Qué es la multicolinealidad.** Que dos o más variables predictoras estén **fuertemente relacionadas entre sí** (no con el target, sino *entre ellas*). Caso extremo: una es función exacta de otras (`charges_day = monthly/30`, o `charges_total ≈ monthly × tenure`).
- **Por qué importa.** En **modelos lineales** (regresión logística/lineal), la multicolinealidad **infla la varianza de los coeficientes**: el modelo no sabe a cuál de las variables correlacionadas atribuir el efecto, así que reparte el crédito de forma inestable. Pequeños cambios en los datos pueden cambiar mucho — incluso de signo— los coeficientes, arruinando la **interpretabilidad**. (El *poder predictivo* suele resistir mejor; lo que se daña es la lectura de "qué variable importa".)
- **Qué es el VIF (Variance Inflation Factor).** Una métrica que cuantifica la multicolinealidad de cada variable: $VIF_j = 1 / (1 - R^2_j)$, donde R^2_j es el R^2 de regresar la variable *j* contra **todas las demás**. Si una variable se puede predecir perfectamente a partir de las otras, $R^2_j \rightarrow 1$ y $VIF_j \rightarrow \infty$.
- **Cómo leer el VIF.** $VIF = 1 \rightarrow$ sin colinealidad. VIF entre 1 y 5 → moderada, normalmente tolerable. $VIF > 5-10 \rightarrow$ problemática: conviene eliminar una variable, combinarlas o regularizar.
- **Cuándo preocuparte.** Sobre todo si tu objetivo es **interpretar** coeficientes, o si usas modelos lineales sin regularización fuerte. Con **árboles/boosting** la multicolinealidad apenas afecta a la predicción (parten por umbrales, no por combinaciones lineales), aunque sí reparte las importancias entre las variables gemelas.
- **Qué pasa si se ignora.** Coeficientes engañosos, importancias diluidas y conclusiones de negocio frágiles ("¿importa `monthly` o `total`?" — el modelo no lo sabe porque son casi la misma cosa).
- **Analogía.** Tener el precio "por mes" y "por día" en la misma tabla: la segunda columna no añade nada, solo confunde a quien reparte el mérito de la predicción. Como dos testigos que repiten la misma frase: no tienes dos pruebas, tienes una dicha dos veces.

REGLA DE DECISIÓN — colinealidad

- Redundancia **exacta** (una variable = función determinista de otra, como `charges_day`) → **elimina una**, sin dudar.
- Colinealidad **fuerte pero no exacta** (corr 0.9996 de `total` vs `monthly×tenure`) → **documentala**; conserva las variables si usas árboles, pero **lee con cautela los coeficientes/importancias**; considera regularización (L2) o PCA si modelas lineal e interpretas.
- Si tu meta es **interpretar** y hay $VIF > 10 \rightarrow$ reduce (elimina, combina o aplica PCA, asumiendo que PCA sacrifica interpretabilidad).

FICHA DE CONCEPTO — Techo de Bayes (error irreducible / Bayes error rate)

- **Qué es.** El **mínimo error que cualquier clasificador posible puede alcanzar** en un problema, dado el conjunto de features disponible. No depende del modelo: es una propiedad de los datos.
- **De dónde sale aquí.** Encontramos 18 perfiles donde **dos (o más) clientes tienen exactamente las mismas features pero desenlaces opuestos** (uno se va, otro se queda). Para esas filas, ningún modelo determinista puede acertar a ambos: si predice "churn" falla con el que se quedó, y viceversa. Ese error es **irreducible** con las variables actuales.

- **Para qué sirve conocerlo.** Para **fixar expectativas realistas**. Si el techo de Bayes implica que el mejor F1 alcanzable ronda ~ 0.64 , perseguir $F1 \approx 0.95$ es perseguir un fantasma: o haces trampa (leakage) o necesitas *features nuevas* (no un modelo más complejo).
- **Cómo bajarlo.** El techo de Bayes solo baja con **mejor información** (nuevas variables que distinguen esos perfiles hoy idénticos), no con modelos más potentes.
- **Qué pasa si se ignora.** Caes en una persecución infinita de décimas de métrica, sobreajustando o sospechando de leakage donde solo hay ruido natural. O peor: confías en un número imposiblemente alto que delata leakage.
- **Analogía.** Si dos pacientes tienen exactamente los mismos síntomas registrados pero uno tiene la enfermedad y el otro no, ningún médico —ni el mejor del mundo— puede acertar siempre con esa ficha. Necesitaría una prueba *adicional* (una feature nueva), no "pensar más fuerte".

📄 FICHA DE CONCEPTO — Desbalance de clases (class imbalance)

- **Qué es.** Que una clase sea mucho más frecuente que la otra. Aquí: 73.4 % "permanece" vs 26.6 % "churn". La clase de interés (churn) es la **minoritaria**.
- **Por qué importa.** (1) Distorsiona métricas ingenuas: con 73/27, "predecir siempre permanece" da 73 % de Accuracy sin aprender nada (ver baselines, §8). (2) Muchos algoritmos, al minimizar el error global, **ignoran** la clase minoritaria porque "sale a cuenta" acertar siempre la mayoría.
- **Cómo se trata.** Tres familias de técnicas: (a) **pesos de clase** (penalizar más los errores en la minoritaria — `class_weight="balanced"`, `scale_pos_weight`); (b) **remuestreo** (sobremuestrear la minoritaria con SMOTE, submuestrear la mayoría); (c) **ajustar el umbral** de decisión y elegir **métricas robustas al desbalance** (F1, MCC, PR-AUC). En este proyecto se usan pesos de clase + umbral + métricas robustas, evitando inventar datos sintéticos.
- **Cuándo NO hace falta hacer nada especial.** Si las clases están razonablemente equilibradas (p. ej. 45/55) o si te importa solo el *ranking* (ROC-AUC) y no la clase asignada.
- **Qué pasa si se ignora.** El modelo "se rinde" con la clase rara (Recall bajísimo), justo la que suele importar (fraude, enfermedad, fuga).
- **Analogía.** Buscar agujas en un pajar: si premias "acertar que es paja", el detector perezoso dice "todo es paja" y acierta el 99 %... sin encontrar una sola aguja.

🔑 REGLA DE DECISIÓN — desbalance

- Clase positiva $< \sim 10\text{--}20\%$ → trata el desbalance: pesos de clase (primera opción, limpia) y/o ajuste de umbral.
- **Nunca** uses Accuracy como métrica principal con clases desbalanceadas → usa **F1, MCC, PR-AUC**.
- Antes de SMOTE, prueba **pesos de clase**: no inventan datos y suelen rendir igual o mejor sin artefactos sintéticos.

⚠ **ERROR TÍPICO — eliminar variables solo por MI/correlación baja.** *Error:* descartar toda variable con $MI \approx 0$ automáticamente. *Consecuencia:* puedes tirar señal que solo existe **en interacción** con otra variable. Por eso aquí solo se elimina `charges_day`, que es una **redundancia probada** (determinista), no una variable de MI baja.

⚠ **ERROR TÍPICO — perseguir un F1 imposible.** *Error:* asumir que con el modelo "correcto" llegarás a $F1 \approx 0.9$ en un problema con techo de Bayes en ~ 0.64 . *Consecuencia:* o sobreajustas, o el número alto que ves esconde leakage. Reconocer el techo evita ambas trampas.

Alternativas a las herramientas de esta fase. VIF para cuantificar multicolinealidad con un número; PCA para descorrelacionar (a costa de interpretabilidad); pruebas χ^2 y V de Cramér para asociación entre categóricas; perfiles de duplicados parciales para estimar el techo de Bayes empíricamente.

☑ QUÉ DEBES RECORDAR (auditoría)

1. **Audita antes de modelar.** Conocer el dataset (tipos, faltantes, cardinalidad, redundancias) evita sorpresas caras.
2. **Mutual Information** ordena variables por señal y capta no-linealidades, pero es univariada: no elimines por MI sola.
3. **Multicolinealidad** daña sobre todo la *interpretación* de modelos lineales; mídela con **VIF** ($> 5\text{--}10$ = problema).
4. El **techo de Bayes** (perfiles idénticos con etiqueta opuesta) fija el máximo rendimiento honesto: un $F1 \sim 0.64$ aquí es coherente, no un fracaso.
5. El **desbalance** (26.6 % churn) obliga a pesos de clase, ajuste de umbral y métricas robustas (F1/MCC/PR-AUC), nunca Accuracy.
6. Elimina solo la redundancia **probada** (`charges_day`); documenta la colinealidad fuerte (`total`) para interpretar con cuidado.

4 · Análisis exploratorio (EDA)

```
df["churn"].value_counts()           # 73.4% / 26.6%
df.groupby("account_contract")["churn"].mean() # mes a mes 42.7%, anual 11.3%, bianual 2.8%
df.groupby("internet_internet_service")["churn"].mean() # fibra 41.9%, DSL 19.0%, sin internet 7.4%
df.groupby("account_paymentmethod")["churn"].mean() # cheque electrónico 45.3% ...
```

Qué se hizo. Un análisis exploratorio dirigido a **cuantificar los drivers** del churn: para cada variable categórica calculamos la **tasa condicional** $P(\text{churn} \mid \text{categoría})$, que nos dice qué grupos se van más.

Hallazgos.

- **Contrato:** mes a mes 42.7 % vs bianual 2.8 % — la variable más discriminante con diferencia (coincide con la MI más alta de la auditoría).
- **Internet:** fibra óptica 41.9 % (probable insatisfacción precio/servicio) vs sin internet 7.4 %.
- **Pago:** cheque electrónico 45.3 % (fricción/segmento), muy por encima de los pagos automáticos ($\sim 15\%$).
- **Tenure (antigüedad):** los clientes nuevos concentran el churn (la curva de densidad de los que evaden está desplazada a la izquierda, es decir, hacia menos meses).

🧠 **INTUICIÓN MATEMÁTICA — la tasa condicional como estimador.** $P(\text{churn} \mid \text{categoría})$ no es más que la media de la variable binaria `churn` dentro de cada grupo. Es un **estimador directo del efecto marginal** de esa categoría. ¿Es fiable? Depende del tamaño del grupo: el error estándar de una proporción es $\sqrt{p(1-p)/n}$. Con n entre ~ 1500 y ~ 3900 por grupo, ese error es de $\sim 0.7\text{--}1.3$ puntos porcentuales. Por eso una diferencia como **2.8 % vs 42.7 %** (¡40 puntos!) es señal real, no ruido: es decenas de errores estándar de distancia.

📄 FICHA DE CONCEPTO — Correlación vs causalidad (y el problema de la confusión)

- **El punto clave.** El EDA encuentra **asociaciones** (correlaciones), no **causas**. "Los que pagan con cheque electrónico se van más" significa que *cheque* y *churn* van juntos, no que el cheque *cause* la fuga.
- **Qué es una variable de confusión (confounder).** Una tercera variable que influye a la vez en la "causa" aparente y en el efecto, creando una asociación espuria. Quizá el cheque electrónico lo usa un **segmento más joven y volátil**: es ese segmento, no el método de pago, lo que explica la fuga.
- **Por qué el EDA es solo el principio.** El EDA es **univariado/bivariado**: mira una variable contra el target sin controlar por las demás. Una tasa alta puede deberse a confusión (la fibra correlaciona con cargos altos, y quizá son los cargos los que empujan la fuga).
- **Cuándo el EDA basta y cuándo no.** Basta para **generar hipótesis** y decidir dónde mirar. **No basta** para afirmar causalidad ni para decisiones del tipo "si cambio X, pasará Y". Para acercarte a eso necesitas modelos multivariados (que controlan por el resto), análisis causal o, idealmente, un experimento (A/B test).
- **Qué pasa si confundes ambas.** Tomas decisiones de negocio equivocadas: "eliminemos el cheque electrónico" podría no reducir el churn ni un punto si el verdadero motor es el segmento de cliente.
- **Analogía.** Las heladerías venden más cuando hay más ahogamientos. ¿El helado causa ahogamientos? No: el **calor** (confounder) provoca ambas cosas. El EDA vería la correlación helado-ahogamiento y se equivocaría si la leyera como causa.

🔊 REGLA DE DECISIÓN — interpretar el EDA

- Usa el EDA para **priorizar hipótesis y features**, nunca para conclusiones causales.
- Si una asociación fuerte podría deberse a una tercera variable → marca esa hipótesis para validarla luego con un modelo multivariado o SHAP (§14), que controla por el resto de variables.

⚠ **ERROR TÍPICO — saltar de correlación a recomendación causal.** *Error:* "los clientes de fibra se van más → quitemos la fibra". *Consecuencia:* acción de negocio costosa y posiblemente inútil si la fibra solo *correlaciona* con el verdadero motor (precio percibido, segmento). El EDA señala **dónde mirar**, no **qué causa qué**.

Alternativas / complementos. Gráficos de mosaico, *Weight of Evidence* (WoE), análisis bivariado con test χ^2 y V de Cramér, gráficos de densidad por clase para variables numéricas.

☑ QUÉ DEBES RECORDAR (EDA)

1. Las tasas condicionales $P(y \mid \text{categoría})$ cuantifican qué grupos disparan el evento; con grupos grandes, diferencias de decenas de puntos son señal real.
2. El EDA produce **correlaciones**, no causas: cuidado con la **confusión**.
3. Es univariado: una tasa alta puede ser efecto de una tercera variable. Validar después con modelos multivariados.
4. Sirve para **formar hipótesis** que el modelo deberá capturar y la interpretabilidad confirmará.

5 · Ingeniería de variables

```
df = df.drop(columns=["account_charges_day"]) # redundante exacta
TARGET = "churn"
CAT = ["internet_internet", "account_contract", "account_paymentmethod"]
NUM = ["customer_tenure", "account_charges_monthly", "account_charges_total"]
BIN = [c for c in df.columns if c not in CAT+NUM+[TARGET]] # ya 0/1
```

Qué se hizo. Aplicamos el principio de **parsimonia**: eliminamos la única redundancia *probada* (*charges_day*) y agrupamos las columnas por **tipo de tratamiento** — numéricas a escalar, categóricas a codificar, binarias a pasar tal cual—. No inventamos variables nuevas sin evidencia de que ayuden.

Por qué tan poca ingeniería (justificación). Cada feature redundante o irrelevante **añade varianza al estimador sin reducir su sesgo**. Esto nos lleva al concepto que vertebra todo el modelado:

📄 FICHA DE CONCEPTO — Trade-off sesgo-varianza (bias-variance tradeoff) (se retoma en §9 y §12)

- **Qué es.** El error de generalización de un modelo se descompone, conceptualmente, en tres partes: **Error = Sesgo² + Varianza + Ruido irreducible**.
- **Sesgo (bias).** Error por suposiciones demasiado rígidas: el modelo es **demasiado simple** para captar el patrón real (p. ej. una recta para datos curvos). Alto sesgo → **underfitting**.
- **Varianza.** Error por sensibilidad excesiva a los datos concretos de entrenamiento: el modelo es **demasiado flexible** y aprende el ruido (memoriza). Alta varianza → **overfitting**.
- **Ruido irreducible.** El **techo de Bayes** (§3): lo que ninguna cantidad de modelado puede eliminar.
- **El "trade-off".** Reducir sesgo (modelo más complejo) suele aumentar varianza, y viceversa. El arte está en encontrar el punto donde la suma es mínima. **Añadir features irrelevantes empuja hacia más varianza** sin bajar el sesgo: peor generalización.
- **Cómo lo conecta esta sección.** Menos features ⇒ menor complejidad ⇒ menor varianza ⇒ menor riesgo de sobreajuste, siempre que no estés tirando señal real. Por eso la ingeniería es mínima y deliberada.
- **Analogía.** Sesgo = un sastre que hace el mismo traje para todos (no se ajusta a nadie). Varianza = un sastre que copia hasta tus arrugas de hoy (mañana, con otra postura, el traje no sirve). Buscas el sastre que capta tu forma sin copiar el ruido del día.

📖 **INTUICIÓN MATEMÁTICA — por qué cada feature de más cuesta.** En un modelo lineal, la varianza de los coeficientes crece con el número de predictores y con la colinealidad entre ellos (recuerda el VIF de §3). Más columnas correlacionadas ⇒ coeficientes más inestables ⇒ predicciones que varían más entre muestras ⇒ peor generalización. La parsimonia no es estética: es control de varianza.

🔊 REGLA DE DECISIÓN — ¿añado esta feature nueva?

- Añádela **solo si mejora la métrica de CV** (no la de train). Si sube train pero no CV → es ruido/overfitting, descártala.
- Si la feature usa **estadísticas globales** del dataset (medias, frecuencias, target) → debe calcularse **dentro del pipeline/fold**, o introduce leakage.

- Con **árboles/boosting**, no crees interacciones manuales sin evidencia: el modelo ya aprende interacciones solo.

⚠ **ERROR TÍPICO — ingeniería de variables como vía de leakage.** *Error:* crear `tasa_churn_por_ciudad` usando la media de churn de **todo** el dataset, o normalizar con estadísticas globales antes del split. *Consecuencia:* leakage grave: la feature ya "sabe" el target. En CV honesta el truco se desinfla, pero si no lo detectas, reportas métricas infladas.

Alternativas que se consideraron y se descartaron por evidencia. `tenure_bins`, `cargos_por_mes_de_vida`, interacciones `contrato × internet`, `target/WoE encoding` como features. Con árboles, las interacciones se aprenden solas; con cardinalidad baja (3–4 categorías), el One-Hot ya captura todo. Ninguna feature nueva demostró mejorar la CV, así que ninguna entró: en ML, "podría ayudar" no es razón suficiente; "mejora la CV de forma estable" sí lo es.

🔑 REGLA DE DECISIÓN — parsimonia (cuándo simplificar)

- A igualdad de rendimiento en CV → menos features y menos complejidad. Cada variable debe **ganarse su sitio**.

Ejemplo intuitivo. Añadir variables "por si acaso" es como llevar 10 mapas distintos a un viaje: más peso, más confusión, y al final usas uno. Mejor un buen mapa que diez mediocres superpuestos.

☑ QUÉ DEBES RECORDAR (ingeniería)

1. **Parsimonia:** cada feature debe justificar su presencia mejorando la **CV**, no el train.
2. El **trade-off sesgo-varianza** explica por qué features de más empeoran la generalización (suben varianza).
3. La ingeniería con **estadísticas globales** es una vía clásica de leakage: hazla dentro del pipeline.
4. Con árboles, las interacciones se aprenden solas; no las crees a mano sin evidencia.
5. Agrupar columnas por tipo (NUM/CAT/BIN) prepara un preprocesamiento limpio y específico por familia de modelo.

6 · Estrategia de validación (FASE 3)

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=RANDOM_STATE) # TEST AISLADO
cv_skf = StratifiedKFold(5, shuffle=True, random_state=RANDOM_STATE)
cv_rskf = RepeatedStratifiedKFold(n_splits=5, n_repeats=3, random_state=RANDOM_STATE)
```

Qué se hizo. Apartamos un **20 % de test que no se vuelve a tocar** hasta la sección 16. **Todas** las decisiones (preprocesamiento, modelo, hiperparámetros, umbral) se toman sobre *train* con **validación cruzada**. Comparamos esquemas de validación y elegimos uno para la fase de comparación de modelos.

Esta sección es el corazón metodológico del proyecto, así que vamos despacio: primero *por qué* existe un test aislado, luego *qué* es la validación cruzada y sus variantes, y por fin *por qué* elegimos Repeated Stratified K-Fold.

📄 FICHA DE CONCEPTO — Train/Test split y "test aislado"

- **Qué es.** Dividir los datos en dos partes: **train** (para aprender y decidir todo) y **test** (apartado, intacto, para medir el rendimiento honesto **una sola vez** al final).
- **Para qué sirve.** El error de un modelo *sobre los datos con los que aprendió* es optimista (los ha visto). El test es una muestra "del futuro": estima cómo rendirá con datos nuevos.
- **Por qué "aislado" (y por qué importa tanto).** En cuanto tomas **cualquier** decisión mirando el test —elegir un modelo, un umbral, un nº de features—, el test deja de ser ciego y su métrica se infla. Por eso aquí se sella en \$6 y no se vuelve a abrir hasta \$16. Era exactamente el error L2 del notebook original (elegía el nº de features mirando el test).
- **Cuándo NO basta un único split.** Con datasets pequeños, un solo test es ruidoso (su métrica depende mucho de qué filas cayeron en él). Ahí entra la validación cruzada (abajo).
- **Variantes según la estructura.** Si hay **tiempo**, el split debe ser temporal (`TimeSeriesSplit`), no aleatorio. Si hay **grupos** (varias filas del mismo cliente), hay que separar por grupo (`GroupKFold`) para que el mismo cliente no esté a la vez en train y test.
- **Qué pasa si se omite.** Sin test reportas el error de entrenamiento como si fuera el de generalización: autoengaño.
- **Analogía.** El test es el examen final a libro cerrado. Si lo "abres" antes para ajustar tu forma de estudiar, deja de medir lo que sabes.

📄 FICHA DE CONCEPTO — Estratificación (stratify)

- **Qué es.** Hacer que cada partición (test, y cada fold) **conservé la misma proporción de clases** que el dataset completo. Con 26.6 % de churn, cada fold tendrá ~26.6 % de churn.
- **Para qué sirve.** Reduce la **varianza** de la estimación y evita que, por azar, un fold quede con muy pocos positivos (lo que dispararía métricas como F1 o Recall en ese fold).
- **Cuándo usarla.** Siempre que la clase esté **desbalanceada** o sea pequeña en términos absolutos. En clasificación desbalanceada es prácticamente obligatoria.
- **Cuándo NO.** En regresión no se estratifica por defecto (aunque existe estratificación por *bins* del target si la distribución es muy sesgada). Si hay estructura temporal, la prioridad es respetar el tiempo, no la proporción.
- **Qué pasa si se omite (en dataset desbalanceado).** Folds con proporciones de clase dispares → métricas inestables entre folds → comparaciones ruidosas y conclusiones poco fiables.
- **Analogía.** Repartir una baraja asegurándote de que cada jugador reciba más o menos la misma cantidad de figuras: si repartes a ciegas, alguna mano queda sin ninguna y "juega" un partido que no representa la realidad.

📄 FICHA DE CONCEPTO — Validación cruzada: K-Fold, Stratified K-Fold y Repeated SKF

- **K-Fold.** Parte el train en *k* trozos (folds). Entrena con *k-1* y evalúa en el restante; repite *k* veces, rotando el fold de evaluación. Resultado: *k* estimaciones del rendimiento, cada dato usado una vez como validación. Promediarlas da una estimación más estable que un solo hold-out.

- **Stratified K-Fold (SKF).** K-Fold + **estratificación**: cada fold mantiene la proporción de clases. Es el estándar en clasificación.
- **Repeated Stratified K-Fold (RSKF).** Repite la SKF varias veces con **barajados distintos**. Con 5 folds × 3 repeticiones obtienes **15 estimaciones** en lugar de 5. Más estimaciones ⇒ media más estable ⇒ menos ruido al comparar.
- **Para qué sirve.** Para estimar el rendimiento **y su variabilidad** sin gastar el test, aprovechando todos los datos de train tanto para entrenar como para validar (en rondas distintas).
- **Cuándo usar cada una.** SKF: clasificación estándar. RSKF: cuando necesitas comparar modelos con poco ruido (como aquí). K-Fold simple (sin estratificar): regresión o clasificación equilibrada.
- **Cuándo NO.** Con datos temporales (usa `TimeSeriesSplit`) o agrupados (usa `GroupKFold`); con datasets enormes donde un solo hold-out ya es suficientemente preciso y la CV sería un coste innecesario.
- **Qué pasa si se omite.** Te quedas con la estimación ruidosa de un único hold-out: no puedes cuantificar la incertidumbre ni comparar modelos con rigor.
- **Analogía.** Medir tu tiempo en 100 m una sola vez (hold-out) vs. correr 15 veces y promediar (RSKF): la media de 15 carreras describe tu nivel real mucho mejor que una sola cronometrada con viento a favor.

Comparación de esquemas (con evidencia del propio proyecto).

Esquema	nº estimaciones	SEM de la media	Uso
Hold-out simple	1	— (no estimable)	descartado
Stratified K-Fold 5	5	mayor	aceptable
Repeated SKF 5×3	15	menor (≈ la mitad)	elegido para comparar
Nested CV	externo×interno	insesgado	estimación final (§13)

Evidencia obtenida (con LogReg): SKF-5 → SEM ≈ 0.0087; **RSKF-5×3** → **SEM ≈ 0.0055**.

 **FICHA DE CONCEPTO — Error estándar de la media (SEM) e intervalo de confianza** *(se retoma en §10)*

- **Qué es el SEM.** Mide cuánto varía la **media** de tus estimaciones si repitieras el experimento. No es la dispersión de los datos (eso es la desviación típica *s*), sino la dispersión de **la media**: $SEM = s / \sqrt{k}$, con *k* = número de estimaciones (folds).
- **Cómo leerlo.** SEM pequeño ⇒ la media que reportas es precisa ⇒ intervalos de confianza estrechos ⇒ puedes detectar diferencias reales entre modelos. SEM grande ⇒ tu media es borrosa y dos modelos "distintos" pueden ser indistinguibles.
- **Qué es un intervalo de confianza (IC 95 %).** Un rango que, con el método usado, contendría la verdadera media el 95 % de las veces si repitieras el muestreo. IC estrecho = estimación precisa.
- **Para qué sirve aquí.** Para no declarar ganador a un modelo por una diferencia que cabe dentro del ruido.
- **Analogía.** El SEM es el "margen de error" de una encuesta: con más encuestados (más folds), el margen se estrecha y puedes afirmar quién va ganando con más seguridad.

 **INTUICIÓN MATEMÁTICA — la raíz de k lo explica todo.** El SEM escala como *s*/√*k*. Pasar de 5 a 15 estimaciones divide el SEM por $\sqrt{(15/5)} = \sqrt{3} \approx 1.73$: casi a la mitad. **No** reduce el ruido a la tercera parte (no es lineal en *k*), sino según la **raíz** de *k*. Por eso duplicar folds da rendimientos decrecientes: pasar de 5 a 15 ayuda mucho; de 100 a 300, poco. La cifra real del proyecto (0.0087 → 0.0055) confirma el factor ~1.73.

Decisiones por evidencia. Para *comparar* modelos usamos **RSKF 5×3** (15 estimaciones, SEM bajo). Para la estimación *final insesgada* usamos **nested CV** (§13). El hold-out simple se descarta para decidir (demasiado ruidoso), y se reserva solo para el test final aislado.

 **FICHA DE CONCEPTO — Nested Cross-Validation (presentación; detalle en §13)**

- **El problema que resuelve.** Si usas la **misma** CV para *elegir* hiperparámetros y para *reportar* el score, ese score es **optimista**: has elegido "el mejor de muchos" sobre esos mismos datos, así que en parte estás midiendo suerte.
- **La solución.** Dos bucles anidados: el **externo** evalúa; el **interno** (dentro de cada fold externo) tunea, usando solo el train de ese fold. Así, la evaluación se hace sobre datos que el tuning **nunca vio**.
- **Por qué es el estándar de oro.** Da una estimación **insesgada** del rendimiento del *procedimiento completo* (incluido el tuning).

 **REGLA DE DECISIÓN — qué esquema de validación elegir**

- Dataset **pequeño/mediano** → validación cruzada (no hold-out único).
- Clasificación **desbalanceada** → **estratifica** siempre.
- Necesitas **comparar modelos con poco ruido** → **Repeated Stratified K-Fold**.
- Vas a **tunar y luego reportar generalización** → **nested CV** para la cifra final.
- Hay **tiempo** → `TimeSeriesSplit`. Hay **grupos** → `GroupKFold`. (No mezclar futuro con pasado ni el mismo grupo en train y test.)

 **ERROR TÍPICO — "arreglar" el leakage con más folds.** *Error:* creer que repetir la CV (más folds) corrige un protocolo con leakage. *Consecuencia:* RSKF reduce **varianza**, no **sesgo**. Si hay leakage, obtienes un número muy estable... y establemente equivocado. El leakage se ataca con **pipelines** (§7), no con más folds.

 **ERROR TÍPICO — test demasiado pequeño o no estratificado.** *Error:* reservar un 5 % de test, o no estratificar con clases desbalanceadas. *Consecuencia:* métrica final ruidosa o sesgada. Aquí, 20 % de 7032 = 1407 filas (≈374 churn) es suficiente y se estratifica.

Alternativas. LOOCV (Leave-One-Out: *k* = *n*; insesgado en sesgo pero de **altísima varianza** y carísimo), `ShuffleSplit` (particiones aleatorias repetidas, con posible solapamiento), `TimeSeriesSplit` (si hubiera orden temporal), Bootstrap .632+ (remuestreo con reemplazo y corrección de optimismo).

 **QUÉ DEBES RECORDAR (validación)**

1. **Aísla el test pronto y tócalo una sola vez.** Cualquier decisión tomada mirándolo lo contamina.
2. **Estratifica** siempre que la clase sea desbalanceada: mantiene la proporción en cada fold y estabiliza las métricas.
3. La **validación cruzada** da estimación + incertidumbre sin gastar el test; **RSKF (5×3)** baja el SEM por $\sqrt{3}$.

4. El **SEM** = $s/\sqrt{k}$: más folds estrechan el intervalo, pero con rendimientos decrecientes (raíz de k).

5. Repetir folds reduce **varianza**, no **sesgo**: el leakage se combate con pipelines, no con más folds.

6. Para la cifra final insesgada tras tunear → **nested CV** (§13).

7 · Preprocesamiento por evidencia (FASE 4)

```
def make_pre(scaler="none", encoder="onehot"):
    scal = {"none": "passthrough", "standard": StandardScaler(), "robust": RobustScaler(),
            "minmax": MinMaxScaler(), "power": PowerTransformer()}[scaler]
    enc = {"onehot": OneHotEncoder(handle_unknown="ignore", drop="if_binary"),
           "ordinal": OrdinalEncoder(...), "target": TargetEncoder(random_state=RANDOM_STATE)}[encoder]
    return ColumnTransformer([("num", scal, NUM), ("cat", enc, CAT), ("bin", "passthrough", BIN)])
```

Qué se hizo. No asumimos qué preprocesamiento es mejor: lo **medimos**. Probamos 5 escaladores con un modelo sensible a la escala (KNN) y 3 codificadores con regresión logística, **todo por CV dentro de pipelines** (para que cada fold ajuste el preprocesador solo con su train, sin leakage).

Evidencia (F1 por CV):

Escalador (KNN)	F1	Codificador (LogReg)	F1
none	0.499	one-hot	0.628
standard	0.549	ordinal	0.629
robust	0.549	target	0.632
power	0.549		
minmax	0.540		

Antes de interpretar la tabla, las fichas de los conceptos en juego.

7.1 · Escalado de variables numéricas

FICHA DE CONCEPTO — Escalado (Standard, Robust, MinMax, Power)

- **Qué es.** Transformar las variables numéricas para que estén en rangos comparables. No cambia el *orden* de los datos, cambia su *escala/forma*.
- **Por qué hace falta (en ciertos modelos).** Algunos algoritmos calculan **distancias** o combinan variables linealmente; si una variable va de 0 a 8700 (*charges_total*) y otra de 0 a 72 (*tenure*), la primera **domina** numéricamente y la segunda casi no cuenta. Escalar las pone en pie de igualdad.
- **Los cuatro escaladores probados:**
 - **StandardScaler** — resta la media y divide por la desviación típica: $z = (x - \mu) / \sigma$. Deja cada variable con media 0 y desviación 1. El más estándar e interpretable ("a cuántas desviaciones de la media estoy").
 - **RobustScaler** — usa **mediana** y **rango intercuartílico** en lugar de media/desviación. Es **robusto a outliers** (un valor extremo no desplaza la mediana como desplaza la media).
 - **MinMaxScaler** — reescala al rango [0, 1] con $(x - \min) / (\max - \min)$. Intuitivo, pero **muy sensible a outliers**: un único valor extremo define el *max* y comprime todo lo demás.
 - **PowerTransformer** — aplica una transformación (Yeo-Johnson/Box-Cox) que **acerca la distribución a una normal**, útil cuando hay mucho sesgo (asimetría).
- **Cuándo escalar.** Modelos basados en **distancia** (KNN, K-Means), **kernels** (SVM-RBF), **lineales con regularización** (la penalización L1/L2 asume escalas comparables) y **redes neuronales** (acelera la convergencia).
- **Cuándo NO escalar.** Modelos basados en **árboles** (Random Forest, boosting): parten por umbrales $x < c$, y un umbral funciona igual de bien en cualquier escala. Escalar árboles es **esfuerzo inútil** (no daña, pero no aporta).
- **Qué pasa si se omite (en modelos de distancia).** La variable de mayor rango domina la métrica y el modelo "ignora" las demás. Aquí se ve clarísimo: KNN sin escalar cae a F1 = 0.499; con StandardScaler sube a 0.549.
- **Analogía.** Comparar personas sumando "diferencia de altura en mm" + "diferencia de edad en años": los milímetros aplastan a los años. Estandarizar pone todo en la misma unidad (desviaciones típicas) para que cada variable pese lo justo.

REGLA DE DECISIÓN — ¿qué escalador?

- Modelo de **distancia/kernel/lineal/red neuronal** → **escala**. Modelo de **árbol/boosting** → **no hace falta**.
- **Outliers presentes** → **RobustScaler** (o Power si además hay mucho sesgo). **Sin outliers** → **StandardScaler** (el más interpretable). **Necesitas rango [0,1] acotado** (p. ej. ciertas redes) → MinMax, pero vigila outliers.
- Aquí: árboles → sin escalado; distancia/lineales → StandardScaler (robust/power empatan, se elige el más estándar).

INTUICIÓN MATEMÁTICA — por qué "none" hunde a KNN. KNN clasifica por **distancia euclídea**: $d = \sqrt{\sum (x_i - x'_i)^2}$. Si una variable tiene rango 8700 y otra 72, la diferencia al cuadrado de la primera es $\sim(8700)^2$ frente a $\sim(72)^2$ de la segunda: la primera aporta $\sim 14\,600\times$ más a la distancia. El modelo "solo ve" *charges_total*. Estandarizar iguala las contribuciones y el F1 sube de 0.499 a 0.549.

7.2 · Codificación de variables categóricas

Las categóricas (*internet_service*, *contract*, *payment_method*) hay que convertirlas en números, pero **cómo** se convierten cambia lo que el modelo puede aprender. Aquí está el concepto que el enunciado pidió explicar con especial detalle.

FICHA DE CONCEPTO — One-Hot Encoding (y por qué no introduce orden falso)

- **Qué es.** Convierte una categórica de k valores en k (o $k-1$) columnas binarias 0/1, una por categoría. Para `contract ∈ {mes_a_mes, anual, bianual}`: tres columnas; un cliente "anual" es `[0, 1, 0]`.
- **Cómo funciona por dentro.** Cada categoría ocupa su propia dimensión. No hay relación numérica entre ellas: "anual" no es "mayor" que "mes a mes", simplemente está en otra columna.
- **Por qué NO introduce un orden artificial.** Si codificaras `{mes_a_mes:0, anual:1, bianual:2}` (Ordinal), el modelo asumiría que `bianual (2)` está "el doble de lejos" de `mes_a_mes (0)` que `anual (1)`, y que `anual` está entre los otros dos en alguna escala. Eso es un **orden inventado** que para variables **nominales** (sin orden real) es falso y confunde a los modelos lineales y de distancia. One-Hot evita esto dándole a cada categoría su propio eje.
- **Qué pasa en modelos lineales.** Con One-Hot, cada categoría obtiene **su propio coeficiente**: el modelo aprende un efecto independiente para "fibra", otro para "DSL", etc. Con Ordinal, el modelo se ve forzado a un **único** coeficiente que multiplica al número de categoría, imponiendo una relación lineal monótona que casi nunca es cierta para nominales.
- **Qué pasa en árboles.** Los árboles toleran mejor el Ordinal (pueden trocear el rango), pero con One-Hot y baja cardinalidad la diferencia es mínima; además One-Hot mantiene la interpretabilidad.
- **Cuándo usarlo.** Categóricas **nominales** (sin orden) de **cardinalidad baja-media** (pocas categorías). Es la opción por defecto, segura e interpretable.
- **Cuándo NO usarlo.** Con **cardinalidad muy alta** (cientos de categorías) genera cientos de columnas dispersas (maldición de la dimensionalidad) → mejor target/frequency encoding. Con categóricas **ordinales** reales (talla $S < M < L$), el Ordinal sí tiene sentido.
- **Detalle `drop="if_binary"`.** Para variables binarias, basta una columna (la otra es su complemento); `drop` evita la redundancia. `handle_unknown="ignore"` hace que una categoría no vista en producción se codifique como todo-ceros en lugar de romper.
- **Qué pasa si se aplica mal.** El error más grave (lo veremos abajo) es aplicarlo a la **variable objetivo**.
- **Analogía.** One-Hot es darle a cada equipo su propia casilla en el marcador. Ordinal sería numerar a los equipos 1, 2, 3 y pretender que el equipo 3 es "tres veces" el equipo 1: un orden inventado que no significa nada.

📄 FICHA DE CONCEPTO — Ordinal Encoding

- **Qué es.** Asigna a cada categoría un entero (`0, 1, 2, ...`).
- **Cuándo usarlo.** Solo cuando existe un **orden real** entre las categorías: tallas ($S < M < L$), niveles educativos, `bajo/medio/alto`. Ahí el número codifica información verdadera.
- **Cuándo NO.** Con categóricas **nominales** (color, método de pago, ciudad): inventa un orden y una distancia que engañan a modelos lineales y de distancia.
- **Qué pasa si se usa mal.** El modelo aprende una relación monótona inexistente; puede empeorar o, peor, parecer que funciona por casualidad del orden alfabético asignado.

📄 FICHA DE CONCEPTO — Target Encoding (y su riesgo de leakage)

- **Qué es.** Reemplaza cada categoría por una estadística del **target** dentro de esa categoría (típicamente la media: "tasa de churn de los clientes con fibra"). Convierte la categórica en un único número informativo.
- **Para qué sirve.** Brilla con **alta cardinalidad** (cientos de categorías), donde One-Hot explotaría en columnas. Una sola columna densa y predictiva.
- **El gran peligro: leakage.** Si calculas la media usando **todo** el dataset (incluida la fila que estás codificando), filtras el target dentro de las features: el modelo "ve" la respuesta. Es una de las formas **más graves** de leakage.
- **Cómo se hace bien.** Con **validación cruzada interna** (out-of-fold) o suavizado bayesiano: la media de cada fila se calcula **sin** esa fila. El `TargetEncoder` de sklearn lo hace por construcción, y debe ir **dentro del pipeline** para que se recalcula en cada fold.
- **Cuándo usarlo.** Alta cardinalidad + necesidad de una representación compacta. Aquí, con cardinalidad 3–4, **no hace falta**: aporta solo +0.004 de F1 sobre One-Hot y sacrifica interpretabilidad.
- **Analogía.** Target encoding bien hecho es como pedir la opinión de un grupo **excluyéndote a ti** para no sesgar la media con tu propia respuesta. Mal hecho, es preguntar "¿qué nota saco?" incluyéndote a ti mismo en el promedio: trampa.

📄 FICHA DE CONCEPTO — Pipeline y ColumnTransformer

- **Qué es un Pipeline.** Un objeto que encadena pasos (preprocesar → modelar) y se comporta como un único modelo: al hacer `fit`, ajusta cada paso en orden; al `predict`, aplica las mismas transformaciones aprendidas.
- **Qué es un ColumnTransformer.** Permite aplicar **distintos** preprocesamientos a **distintos grupos de columnas** (escalar las numéricas, codificar las categóricas, pasar las binarias tal cual) en un solo objeto.
- **Para qué sirven (la razón de fondo).** Para que el preprocesamiento se ajuste **dentro de cada fold**, usando solo el train de ese fold. Esto **elimina el leakage de proceso por construcción**: el scaler/encoder nunca ve los datos de validación/test al aprender sus parámetros.
- **Cuándo usarlos.** Siempre que haya preprocesamiento con parámetros aprendidos (escalado, codificación, imputación, selección). Es decir, casi siempre.
- **Qué pasa si se omite.** El error L1 del notebook original: `fit_transform` sobre todo X antes del split → el scaler aprende con datos de test → leakage. Además, sin pipeline el flujo es frágil y no reproducible célula a célula.
- **Bonus en despliegue.** Empaquetar todo en un pipeline elimina el *training-serving skew* (§17): en producción aplicas exactamente las mismas transformaciones.
- **Analogía.** Un pipeline es una cadena de montaje sellada: metes datos crudos por un extremo y sale una predicción por el otro, con todas las piezas calibradas juntas y de forma consistente.

Decisiones por evidencia.

- **Árboles** → **sin escalado** (invariantes a transformaciones monótonas).
- **Modelos de distancia/lineales** → **StandardScaler** (robust y power empatan en F1; se elige el más estándar e interpretable).
- **Codificador** → **One-Hot**: el target encoding solo aporta +0.004 con cardinalidad 3–4; One-Hot es interpretable y sin riesgo de leakage. La diferencia es ruido, así que gana la opción simple y transparente.

🔑 REGLA DE DECISIÓN — codificación de categóricas

- **Nominal + cardinalidad baja/media** → **One-Hot**.
- **Ordinal real** (hay orden) → **Ordinal**.
- **Cardinalidad alta** (decenas/cientos) → **Target encoding** (con CV interna) o **frequency encoding**.
- Ante un empate técnico entre encoders → elige el **más simple e interpretable** (aquí, One-Hot).

⚠ **ERROR TÍPICO — One-Hot sobre la variable objetivo.** *Error:* aplicar One-Hot (o cualquier expansión) a la **target** de un problema de clasificación binaria. *Consecuencia:* dejas de resolver una clasificación estándar; el problema se transforma en otro distinto (multi-salida) y las métricas dejan de tener el significado que crees. La target binaria se codifica como **0/1**, nada más.

⚠ **ERROR TÍPICO — escalar/codificar antes de separar train/test.** *Error:* `scaler.fit_transform(X)` o `encoder.fit_transform(X)` sobre **todo** el dataset, y luego hacer el split (el error L1 del notebook original). *Consecuencia:* **data leakage**. El scaler aprende min/max/media usando también el test; con `StandardScaler`, `PowerTransformer`, imputación por media o *target encoding* el efecto puede ser grave. **Solución:** todo dentro del **Pipeline**, que se reajusta por fold.

⚠ **ERROR TÍPICO — target encoding con la media global.** *Error:* `df["cat"].map(df.groupby("cat")["y"].mean())` usando todo el dataset. *Consecuencia:* leakage extremadamente grave: cada fila lleva información de su propio target. **Solución:** versión con CV interna (out-of-fold), dentro del pipeline.

Alternativas. `QuantileTransformer` (mapea a una distribución uniforme/normal por cuantiles, muy robusto), *binning* supervisado, *Weight of Evidence* (WoE, popular en scoring crediticio), *frequency encoding* (cada categoría → su frecuencia; útil con cardinalidad alta, irrelevante aquí).

☑ QUÉ DEBES RECORDAR (preprocesamiento)

1. **Decide por evidencia (CV), no por costumbre:** prueba escaladores/codificadores dentro de pipelines.
2. **Escala** modelos de distancia/kernel/lineales; **no escales** árboles (es inútil).
3. **One-Hot** para nominales de baja cardinalidad: no inventa orden y da un coeficiente por categoría. **Ordinal** solo si hay orden real. **Target** para alta cardinalidad, siempre con CV interna.
4. **Pipelines + ColumnTransformer eliminan el leakage de proceso por construcción** (cada fold reajusta el preprocesador). Úsalos siempre.
5. El leakage más grave del preprocesamiento es **target encoding con media global** y **escalar antes del split**.
6. Ante empates entre opciones de preprocesamiento, gana la **más simple e interpretable**.

8 · Baselines (FASE 5)

```
DummyClassifier(strategy="most_frequent") # predice siempre "no churn"
DummyClassifier(strategy="stratified")    # predice al azar respetando proporciones
LogisticRegression(class_weight="balanced") # baseline FUERTE y candidato serio
```

Qué se hizo. Establecimos una **línea de flotación**: tres baselines contra los que todo modelo complejo debe demostrar su valía. Si un modelo sofisticado no supera con claridad a un baseline tonto, no justifica su coste.

Resultados.

- **Mayoría (most_frequent):** predice siempre "no churn". Accuracy $\approx 73\%$ (¡engañoso!) pero **Recall = 0** y **F1 = 0** sobre la clase churn: no detecta una sola fuga.
- **Estratificado (stratified):** predice al azar respetando proporciones → F1 ≈ 0.27 ($\approx$ la tasa base), MCC ≈ 0 .
- **Regresión logística balanceada:** baseline **fuerte** que —spoiler— terminará siendo el modelo final.

📄 FICHA DE CONCEPTO — Baseline e hipótesis nula

- **Qué es un baseline.** Un modelo deliberadamente simple (o tonto) que fija el **mínimo a superar**. Hay dos niveles: *baselines triviales* (Dummy: mayoría, azar) y *baselines fuertes* (un modelo simple bien hecho, como la LogReg).
- **Qué es la hipótesis nula aquí.** "No hay señal aprendible". El `DummyClassifier` la **materializa**: si tu modelo no la supera de forma significativa, no tienes evidencia de que haya aprendido nada.
- **Para qué sirve.** (1) Detectar métricas engañosas (Accuracy alta del Dummy de mayoría). (2) Cuantificar cuánto aporta *de verdad* un modelo complejo sobre lo trivial. (3) Decidir si la complejidad merece la pena.
- **Cuándo usarlo.** Siempre, al principio del modelado. Es barato y orienta todo lo demás.
- **Qué pasa si se omite.** Celebras métricas que un modelo tonto igualaría; tomas por "buena" una Accuracy del 73 % que es pura tasa base.
- **Analogía.** Un detector de fraude que dice "nunca hay fraude" acierta el 99.9 % de las veces... y es inútil. El baseline de mayoría es ese detector: te recuerda que "acertar mucho" no es "servir para algo".

🧠 **INTUICIÓN MATEMÁTICA — por qué Accuracy miente aquí.** Accuracy = (aciertos)/(total). Con 73 % de clase "permanece", predecir **siempre** "permanece" da Accuracy = 0.73 sin mirar una sola variable. La métrica premia la **inacción** en problemas desbalanceados. Por eso el baseline de mayoría es tan didáctico: pone un 0.73 sobre la mesa y obliga a preguntar "¿mi modelo realmente aprende, o solo iguala a la moneda cargada?". La respuesta honesta la dan F1, MCC y PR-AUC, que valen ≈ 0 para ese Dummy.

🔑 REGLA DE DECISIÓN — baselines

- Empieza **siempre** por un Dummy (mayoría + estratificado) y un modelo lineal simple.
- Si tu modelo complejo **no** supera al baseline fuerte de forma significativa → no uses el complejo.
- En desbalance, mira el F1/MCC del baseline, **no** su Accuracy.

⚠ **ERROR TÍPICO — confundir Accuracy alta del Dummy con un buen modelo.** *Error:* reportar "73 % de accuracy" como logro. *Consecuencia:* presentas como éxito lo que un `DummyClassifier` consigue sin aprender. La defensa es reportar F1, MCC y PR-AUC junto a (o en lugar de) Accuracy.

Alternativas. Baseline por **reglas de negocio** (p. ej. "todo cliente mes-a-mes con tenure < 6 es churn"), o un *prior* bayesiano. Un buen baseline de reglas a veces es sorprendentemente competitivo y es la verdadera vara de medir del negocio.

☑ QUÉ DEBES RECORDAR (baselines)

1. El baseline es el **piso cuantitativo**: todo modelo serio debe superarlo con claridad.
2. El Dummy de mayoría **desenmascara la Accuracy** en problemas desbalanceados (73 % "gratis", F1 = 0).

3. Usa baselines triviales (Dummy) y uno fuerte (lineal): a veces el fuerte ya es el ganador (¡aquí lo será!).

9 · Modelado: zoo de modelos (FASE 6)

```
spw = (y_train==0).sum() / (y_train==1).sum() # ≈ 2.77, peso de la clase churn
modelos = { "LogReg", "SVM-RBF", "KNN", "RandomForest", "ExtraTrees",
            "HistGB", "XGBoost", "LightGBM", "CatBoost" } # cada uno en su Pipeline
# Evaluación: cross_validate(pipe, X_train, y_train, cv=cv_rskf, scoring=SCORERS)
```

Qué se hizo. Entrenamos **nueve familias** de modelos que cubren el espacio de hipótesis: lineal (LogReg), kernel (SVM-RBF), basado en instancias (KNN), *bagging* de árboles (Random Forest, ExtraTrees) y *boosting* de gradiente (HistGB, XGBoost, LightGBM, CatBoost). Cada modelo va en **su** pipeline con el preprocesamiento adecuado (§7) y su manejo del desbalance.

📄 FICHA DE CONCEPTO — Class weight y scale_pos_weight (manejar el desbalance)

- **Qué es.** Decirle al modelo que un error en la clase **minoritaria** cuesta más que uno en la mayoritaria. Modifica la **función de pérdida**, no los datos.
- **Las tres formas usadas:** `class_weight="balanced"` (LogReg, SVM, RF, ExtraTrees, HistGB) asigna pesos inversamente proporcionales a la frecuencia de clase; `scale_pos_weight=spw` (XGBoost, LightGBM) multiplica el gradiente de la clase positiva por `spw ≈ 2.77`; `auto_class_weights="Balanced"` (CatBoost) hace lo equivalente.
- **Para qué sirve.** Que el modelo **no ignore** la clase rara. Sin pesos, minimizar el error global empuja a "predecir siempre la mayoría".
- **Cuándo usarlo.** Clases desbalanceadas y te importa la minoritaria. Es la primera opción (limpia, no inventa datos).
- **Cuándo NO / con cuidado.** Si el desbalance es leve, o si te importa solo el *ranking* (ROC-AUC). Ojo: pesos agresivos **descalibran** las probabilidades (las inflan hacia la clase positiva) y, con umbral 0.5, suben Recall a costa de Precisión — justo lo que le pasa a Random Forest/ExtraTrees aquí.
- **Alternativa.** Remuestreo (SMOTE/undersampling). Los pesos suelen ser preferibles: no crean datos sintéticos que pueden introducir artefactos.
- **Analogía.** Es subir el volumen de la voz minoritaria en una reunión para que no la aplaste la mayoría, en vez de clonar personas que opinen como ella (eso sería SMOTE).

📄 FICHA DE CONCEPTO — Bagging vs Boosting (dos formas de combinar árboles)

- **El problema común.** Un árbol de decisión solo es inestable (alta varianza): pequeños cambios en los datos cambian mucho el árbol. Los *ensembles* combinan muchos árboles para corregirlo, por dos caminos opuestos.
- **Bagging (Random Forest, ExtraTrees).** Entrena muchos árboles **en paralelo**, cada uno sobre una muestra bootstrap y un subconjunto de variables, y **promedia**. Reduce **varianza** (suaviza la inestabilidad). Cada árbol es profundo (bajo sesgo, alta varianza) y el promedio cancela el ruido.
- **Boosting (HistGB, XGBoost, LightGBM, CatBoost).** Entrena árboles **en secuencia**, cada uno corrigiendo los errores del anterior. Reduce **sesgo** (y algo de varianza con regularización). Muy potente, pero **propenso a sobreajustar** si no se regulariza/para a tiempo.
- **Cuándo cada uno.** Bagging: robusto, pocas manos, buen punto de partida. Boosting: suele dar el mejor rendimiento bruto en datos tabulares **cuando hay patrón no lineal** que explotar y se tunea con cuidado.
- **Por qué aquí no ganan.** El problema es casi linealmente separable en las variables clave; la flexibilidad extra de los árboles no compra exactitud, solo varianza y riesgo de sobreajuste (lo veremos en la tabla [gap](#)).
- **Analogía.** Bagging = pedir a 100 expertos independientes su opinión y promediar (cancela errores idiosincráticos). Boosting = un experto que repasa una y otra vez los casos donde falló, especializándose... hasta que, si se obsesiona, memoriza las erratas del examen de práctica.

📄 FICHA DE CONCEPTO — Overfitting, underfitting y el gap train-CV

- **Overfitting (sobreajuste).** El modelo aprende el **ruido** del train, no solo el patrón. Síntoma: rinde de maravilla en train y mal en validación. Es **alta varianza**.
- **Underfitting (subajuste).** El modelo es demasiado simple para captar el patrón. Síntoma: rinde mal en train y en validación. Es **alto sesgo**.
- **El gap train-CV.** La diferencia `F1_train - F1_CV` es el **termómetro del sobreajuste**. `Gap ≈ 0` → el modelo generaliza lo que aprende. `Gap grande` → memoriza (lo que sabe en train no se traslada a datos nuevos).
- **Cómo leerlo aquí.** LogReg: gap +0.003 (no memoriza nada). RandomForest: gap +0.447 con `F1_train ≈ 0.997` (memoriza casi todo el train). XGBoost +0.125, HistGB +0.178, LightGBM +0.252 (memorización creciente).
- **Qué hacer ante overfitting.** Reducir complejidad (menos profundidad, menos features), **regularizar** (L1/L2, λ), conseguir más datos, o early stopping.
- **Qué hacer ante underfitting.** Aumentar complejidad, añadir features informativas, reducir regularización.
- **Analogía.** Overfitting = el estudiante que memoriza las respuestas del examen de práctica y se hunde con preguntas nuevas. Underfitting = el que estudió tan por encima que falla hasta las de práctica.

📄 REGLA DE DECISIÓN — diagnóstico por el gap

- **Train $\gg$ CV** (gap grande) → **overfitting** → simplifica/regulariza/más datos.
- **Train $\approx$ CV pero ambos bajos** → **underfitting** → más complejidad/mejores features.
- **Train $\approx$ CV y ambos altos** → buen punto; cerca del techo de Bayes, no fuerces más.

Antes de la tabla de resultados, necesitamos entender las métricas con las que se mide. (Aquí van las fichas esenciales; el tratamiento exhaustivo y la comparación con regresión están en el §20.)

📄 FICHA DE CONCEPTO — Matriz de confusión: Precision, Recall, F1, Accuracy

- **La matriz de confusión** cruza realidad vs predicción en 4 casillas: **VP** (verdaderos positivos), **FP** (falsos positivos), **VN** (verdaderos negativos), **FN** (falsos negativos).
- **Precision = $VP/(VP+FP)$.** De los que predije churn, ¿cuántos lo eran? Penaliza **falsas alarmas**. Baja precisión = molestas/incentivas a clientes que no se iban.

- **Recall (sensibilidad) = $VP/(VP+FN)$.** De los que de verdad hicieron churn, ¿cuántos detecté? Penaliza **fugas no detectadas**. Bajo recall = se te escapan clientes que se iban.
- **F1 = media armónica de precision y recall = $2 \cdot P \cdot R / (P+R)$.** Equilibra ambos: solo es alta si **los dos** lo son. Usa la media *armónica* (no la aritmética) precisamente para castigar el desequilibrio: si una es 0.9 y la otra 0.1, la media armónica ≈ 0.18 , no 0.5.
- **Accuracy = $(VP+VN)/total$.** Proporción global de aciertos. **Engañosa con desbalance** (el baseline ya lo demostró).
- **Cuándo cada una.** Recall importa si un FN es muy caro (cáncer no detectado, fuga no prevenida). Precision importa si un FP es caro (intervención costosa, alarma molesta). F1 cuando ambos importan y hay desbalance. Accuracy solo con clases equilibradas.
- **Analogía (pesca con red).** Recall = qué fracción de los peces del lago atrapé. Precision = qué fracción de lo que subí en la red eran peces (y no botas viejas). F1 = un único número que premia pescar muchos peces y poca basura.

 **FICHA DE CONCEPTO — MCC (coeficiente de correlación de Matthews)**

- **Qué es.** Una correlación entre las etiquetas reales y las predichas, en $[-1, +1]$. $+1$ = predicción perfecta, 0 = como el azar, -1 = predicción sistemáticamente invertida.
- **Por qué es la más honesta bajo desbalance.** Usa **las cuatro** casillas de la matriz (VP, VN, FP, FN). Solo es alto si el modelo acierta en **ambas** clases. No se deja engañar por "acertar siempre la mayoritaria" (eso da $MCC \approx 0$).
- **Cuándo usarla.** Como métrica robusta de cabecera en clasificación desbalanceada, junto a F1.
- **Cómo leerla.** Aquí, $MCC \approx 0.48$ indica una correlación moderada real entre predicción y verdad (muy por encima del 0 del Dummy), coherente con un problema que tiene techo de Bayes.
- **Analogía.** Es la "nota global honesta" que solo sube si haces bien las dos partes del examen, no si bordas una y abandonas la otra.

 **FICHA DE CONCEPTO — ROC-AUC y PR-AUC (métricas independientes del umbral)**

- **Idea común.** Un clasificador produce una **probabilidad**; el umbral decide la clase. Estas métricas evalúan la calidad del **ranking** de probabilidades a **todos** los umbrales, sin fijar uno.
- **ROC-AUC.** Área bajo la curva TPR (recall) vs FPR (falsos positivos sobre los negativos). Probabilidad de que el modelo puntúe más alto a un positivo aleatorio que a un negativo aleatorio. 0.5 = azar, 1.0 = perfecto.
- **PR-AUC (average precision).** Área bajo la curva Precision vs Recall. Se centra en la **clase positiva**.
- **Cuándo cada una.** Con **desbalance**, **PR-AUC es más informativa** que ROC-AUC: la ROC puede parecer optimista porque su eje FPR se "diluye" con muchísimos negativos, mientras que la precisión "siente" cada falso positivo. ROC-AUC es buena para comparar capacidad de ranking en general.
- **Cómo leerlas.** ROC-AUC ≈ 0.84 aquí = buen poder de ordenación. PR-AUC ≈ 0.65 = razonable dado que la clase positiva es solo el 26.6% (el "azar" de PR-AUC es la tasa base ≈ 0.27 , no 0.5).
- **Analogía.** ROC-AUC mide si sabes **ordenar** clientes de más a menos arriesgados. PR-AUC mide si, además, los más arriesgados según el modelo **realmente** se van (precisión en la cima del ranking).

Resultados reales de la ejecución (RSKF 5×3, ordenados por F1; **gap** = $F1_{train} - F1_{cv}$ = señal de sobreajuste):

Modelo	F1	MCC	ROC-AUC	PR-AUC	F1_train	gap
LogReg	0.6313	0.4804	0.8443	0.6525	0.6345	+0.003
CatBoost	0.6293	0.4767	0.8438	0.6610	0.6994	+0.070
SVM-RBF	0.6287	0.4757	0.8271	0.5992	0.6581	+0.029
XGBoost	0.6246	0.4698	0.8393	0.6539	0.7494	+0.125
HistGB	0.6215	0.4663	0.8349	0.6442	0.7993	+0.178
LightGBM	0.6125	0.4563	0.8285	0.6314	0.8648	+0.252
KNN	0.6036	0.4684	0.8316	0.6088	0.6297	+0.026
RandomForest	0.5493	0.4192	0.8260	0.6231	0.9965	+0.447
ExtraTrees	0.5334	0.3946	0.7975	0.5699	0.9961	+0.463

Hallazgo central (doble). (1) La **Regresión Logística obtiene el F1/MCC más alto**, por encima de todos los boosters. (2) Más importante: su **gap train–CV es ~ 0.003** , mientras los boosters memorizan el train (XGBoost $+0.125$, HistGB $+0.178$, LightGBM $+0.252$, **RandomForest $+0.447$** con $F1_{train} \approx 0.997$). El problema es, en gran medida, **linealmente separable** en sus variables clave (contrato, tenure, cargos): la flexibilidad extra de los árboles no compra exactitud, solo **varianza y riesgo de sobreajuste**.

Justificación estadística. El *trade-off* sesgo–varianza (§5) explica todo: LogReg tiene **sesgo moderado / varianza baja**; los boosters, **sesgo bajo / varianza alta**. Cuando la frontera real es casi lineal, el sesgo de LogReg no penaliza y su baja varianza gana. La columna **gap** lo cuantifica: un modelo que saca 0.997 en train y 0.549 en CV (RandomForest) está **memorizando**, no aprendiendo. Random Forest/ExtraTrees además caen en F1 porque con **class_weight=balanced** y umbral 0.5 sesgan hacia la clase positiva, hinchando recall a costa de precisión (su ROC-AUC sigue decente: **ordenan** bien, pero el umbral 0.5 no es su punto óptimo).

 **ERROR TÍPICO — declarar ganador por milésimas de F1.** Error: "LogReg (0.6313) > CatBoost (0.6293), gana LogReg por rendimiento". Consecuencia: sobreinterpretas ruido de muestreo. La diferencia 0.6313 vs 0.6293 hay que **contrastarla estadísticamente** (§10) antes de afirmar nada. (La conclusión correcta no es "LogReg gana por F1", sino "LogReg empató y además es la más simple y estable".)

 **ERROR TÍPICO — comparar solo con umbral 0.5.** Error: juzgar modelos únicamente por F1 al umbral por defecto. Consecuencia: penalizas injustamente a modelos con probabilidades mal calibradas que ordenarían bien con otro umbral. Por eso miramos también **ROC-AUC y PR-AUC** (independientes del umbral).

Alternativas. *Stacking/voting* de los mejores modelos; *balanced bagging*; remuestreo SMOTE (no usado: los pesos de clase son más limpios y no inventan datos sintéticos que pueden introducir artefactos).

Ejemplo intuitivo. Para cortar una hoja de papel en línea recta, una regla (LogReg) funciona mejor que un brazo robótico de 7 ejes (boosting): el robot puede hacer curvas que no necesitas y **tiembla** más (más varianza). La complejidad solo paga si la tarea la exige.

☑

QUÉ DEBES RECORDAR (modelado)

1. Prueba **familias diversas** (lineal, kernel, instancias, bagging, boosting) con el mismo protocolo.
2. El **gap train-CV** es tu termómetro de sobreajuste: $F1_{train} \approx 1.0$ con CV mediocre = memorización.
3. **Más complejo ≠ mejor**. Si el patrón es casi lineal, LogReg gana por baja varianza (gap +0.003 vs +0.447 de RF).
4. **Pesos de clase** manejan el desbalance sin inventar datos, pero **descalibran** probabilidades (afectan al umbral 0.5).
5. Mira **F1 y MCC** (dependen del umbral) **junto a ROC-AUC y PR-AUC** (no dependen del umbral) para un juicio completo.
6. Una diferencia de F1 de milésimas **no es** una victoria: hay que contrastarla (\$10).

10 · Estabilidad estadística (FASE 8)

```
def ic95(scores):
    m, sem = np.mean(scores), stats.sem(scores)
    return stats.t.interval(0.95, len(scores)-1, loc=m, scale=sem)
# Wilcoxon pareado entre el mejor modelo y cada rival sobre los 15 folds
stats.wilcoxon(f1_folds[best], f1_folds[other])
```

Qué se hizo. Para cada modelo reportamos media, desviación e **IC 95 %** del F1 sobre los 15 folds, y contrastamos al mejor (LogReg) contra cada rival con un **test de Wilcoxon pareado**. Los modelos con $p > 0.05$ son **estadísticamente indistinguibles** del mejor. Este paso responde a la pregunta crítica: *¿la diferencia de F1 entre modelos es señal o es ruido de muestreo?*

Resultados reales.

Modelo	F1	IC95	p (vs LogReg)	¿≈ mejor?
LogReg	0.6313	[0.6230, 0.6396]	— (referencia)	—
CatBoost	0.6293	[0.6197, 0.6389]	0.489	☑ sí
SVM-RBF	0.6287	[0.6201, 0.6374]	0.303	☑ sí
XGBoost	0.6246	[0.6145, 0.6346]	0.135	☑ sí
HistGB	0.6215	[0.6141, 0.6290]	0.048	✗ (al límite)
LightGBM	0.6125	[0.6014, 0.6236]	0.0001	✗ no
KNN	0.6036	[0.5927, 0.6145]	0.0006	✗ no
RandomForest	0.5493	[0.5380, 0.5606]	0.0001	✗ no
ExtraTrees	0.5334	[0.5193, 0.5475]	0.0001	✗ no

Lectura. CatBoost, SVM y XGBoost **no se distinguen** estadísticamente de LogReg ($p > 0.05$). Entre todos ellos, LogReg es el más simple, el más estable (menor gap) y el más interpretable → la REGLA FINAL lo elige sin ambigüedad.

📄

FICHA DE CONCEPTO — Test de Wilcoxon pareado (de los rangos con signo)

- **Qué es.** Una prueba estadística **no paramétrica** que decide si dos conjuntos de mediciones **pareadas** difieren de forma sistemática. "Pareadas" = cada medición de A tiene su correspondiente de B sobre **el mismo caso** (aquí, el mismo fold).
- **Por qué pareado.** Los 15 folds son **las mismas particiones** para todos los modelos. Eso permite restar fold a fold y **eliminar la varianza** debida a "qué datos cayeron en cada fold", aislando la diferencia entre modelos. Comparar sin parear (como si fueran muestras independientes) desperdiciaría esta ventaja y tendría menos potencia.
- **Por qué no paramétrico (Wilcoxon en vez de t-test).** El t-test pareado asume que las **diferencias** son aproximadamente normales. Con 15 muestras y posibles colas, esa suposición es frágil. Wilcoxon usa los **rangos** de las diferencias (no sus valores exactos), así que no asume normalidad: es más robusto en este régimen.
- **Qué es el p-valor.** La probabilidad de observar una diferencia **al menos tan grande** como la tuya **si en realidad no hubiera diferencia** (hipótesis nula). p pequeño (< 0.05) → improbable que sea casualidad → diferencia significativa. p grande → no hay evidencia de diferencia.
- **Qué NO significa un $p > 0.05$.** No "prueba que son iguales"; significa "no hay evidencia suficiente de que difieran". La diferencia entre "no hay evidencia de diferencia" y "evidencia de que no hay diferencia" es sutil pero crucial.
- **Cuándo usarlo.** Para comparar dos modelos sobre los mismos folds de CV. Para comparar muchos, ojo a las comparaciones múltiples (abajo).
- **Analogía.** Dos corredores con tiempos medios 10.10 s y 10.12 s, pero cuya variación día a día es ± 0.15 s: decir "el primero es más rápido" es leer el ruido. Wilcoxon es el juez que, mirando carrera a carrera (pareado), dicta "empate técnico".

📖 INTUICIÓN MATEMÁTICA — el IC 95 % con t de Student. El intervalo es $\text{media} \pm t \cdot \text{SEM}$, donde t es el valor de la distribución t de Student para 14 grados de libertad (15 folds $- 1$). Se usa la **t**, no la normal, porque con pocas muestras la incertidumbre sobre la propia desviación engorda las colas. Leer los IC de la tabla es directo: si el IC de un rival **se solapa** ampliamente con el de LogReg (p. ej. CatBoost [0.6197, 0.6389] vs LogReg [0.6230, 0.6396]), sus rendimientos son compatibles; si está claramente por debajo y sin solape (ExtraTrees [0.5193, 0.5475]), es peor de verdad.

📄

FICHA DE CONCEPTO — Comparaciones múltiples (Bonferroni/Holm)

- **El problema.** Cada test al 5 % tiene un 5 % de probabilidad de un **falso positivo** (declarar diferencia donde no la hay). Si haces 8 comparaciones, la probabilidad de **al menos un** falso positivo sube muy por encima del 5 % ($\approx 1 - 0.95^8 \approx 34\%$). Cuantos más tests, más fácil "encontrar" una diferencia que

es azar.

- **Las correcciones. Bonferroni** divide el umbral entre el nº de tests ($0.05/8 \approx 0.006$): conservador pero simple. **Holm** es una versión secuencial, igual de segura pero con más potencia.
- **Cuándo aplicarlas.** Siempre que hagas varias comparaciones y vayas a sacar una conclusión "fuerte" de una de ellas.
- **Por qué aquí no cambia el veredicto.** La decisión final favorece la **simplicidad**: aunque corrigiéramos, LogReg seguiría siendo elegible (empata con los mejores) y sería el preferido por parsimonia. Se documenta por rigor.
- **Analogía.** Si tiras los dados muchas veces, tarde o temprano sale un doble seis "sorprendente". Las correcciones ajustan el listón de "sorpresa" según cuántas veces tiraste.

🔑 REGLA DE DECISIÓN — ¿hay un ganador de verdad?

- Diferencia de métrica + **IC que se solapan** o **p > 0.05** → **empate técnico** → no prefieras el más complejo; elige por parsimonia/estabilidad/interpretabilidad.
- Diferencia con **p < 0.05** (y, si hay muchos tests, tras corrección) e IC separados → diferencia **real** → puedes preferir al ganador por rendimiento.
- Folds compartidos → usa un test **pareado** (Wilcoxon o t pareado), no uno de muestras independientes.

⚠ **ERROR TÍPICO — anunciar mejoras sin test estadístico.** *Error:* "mi modelo nuevo sube el F1 de 0.631 a 0.633, ¡mejora!". *Consecuencia:* publicas/despliegas ruido. Sin IC ni test pareado, una diferencia de milésimas sobre una partición no es defendible (era el defecto M2 del notebook original: comparar RF vs XGB en una sola partición).

⚠ **ERROR TÍPICO — interpretar p > 0.05 como "son idénticos".** *Error:* "p = 0.49 → CatBoost y LogReg son iguales". *Consecuencia:* sobre-afirmas. Lo correcto: "no hay evidencia de que difieran". Wilcoxon con 15 pares tiene **potencia limitada**; basta para no preferir lo complejo sin evidencia, no para demostrar igualdad.

Alternativas. Test de **McNemar** sobre las predicciones del test (compara errores discordantes entre dos modelos en el mismo conjunto); **t-test corregido de Nadeau-Bengio** (ajusta la dependencia entre folds de CV, que viola la independencia que asume el t-test clásico); **comparación bayesiana de clasificadores** (estima la probabilidad de que A > B, en lugar de un p-valor).

☑ QUÉ DEBES RECORDAR (estadística)

1. Una diferencia de métrica **no es** una mejora hasta que sobrevive a un **test estadístico**.
2. Folds compartidos → test **pareado** (Wilcoxon): elimina la varianza de "qué datos cayeron en cada fold".
3. **Wilcoxon** (no t-test) porque no asume normalidad de las diferencias, apropiado con 15 muestras.
4. **p > 0.05** = "sin evidencia de diferencia", **no** "son iguales". Es suficiente para no preferir lo complejo.
5. Con **muchas** comparaciones, corrige (Bonferroni/Holm) para no cazar falsos positivos.
6. Aquí, CatBoost/SVM/XGBoost empatan con LogReg → la decisión la toma la parsimonia, no el F1.

11 · Optimización de hiperparámetros con Optuna (FASE 7)

```
study = optuna.create_study(direction="maximize",
    sampler=optuna.samplers.TPESampler(seed=RANDOM_STATE),
    pruner=optuna.pruners.MedianPruner(n_startup_trials=5, n_warmup_steps=2))
# objetivo: F1 medio por CV, reportando valor parcial por fold -> pruning
# boosters: early stopping (HistGB interno; LightGBM con eval_set + callbacks)
```

Qué se hizo. Sustituimos **GridSearchCV** (búsqueda exhaustiva, ciega) por **optimización bayesiana** (TPE) con dos aceleradores —**pruning** y **early stopping**— y optimizamos tres representantes: LogReg, HistGB y LightGBM.

📄 FICHA DE CONCEPTO — Hiperparámetros y por qué se "tunean"

- **Parámetros vs hiperparámetros.** Los **parámetros** los aprende el modelo de los datos (los coeficientes de la regresión, los cortes de los árboles). Los **hiperparámetros** los fijas tú **antes** de entrenar (la regularización C, la profundidad del árbol, la tasa de aprendizaje). Controlan la **capacidad** del modelo y, por tanto, el equilibrio sesgo-varianza.
- **Por qué importan.** El mismo algoritmo con hiperparámetros distintos puede infraajustar, ajustar bien o sobreajustar. "Tunear" es buscar la combinación que mejor generaliza (medida por CV).

📄 FICHA DE CONCEPTO — Optimización bayesiana / TPE (Optuna) vs Grid/Random Search

- **Grid Search.** Prueba **todas** las combinaciones de una rejilla. Sufre la **maldición de la dimensionalidad**: 4 hiperparámetros × 5 valores = 625 combinaciones, casi todas malas. Ciego: no aprende de lo ya probado.
- **Random Search.** Muestrea combinaciones al azar. Sorprendentemente competitivo (suele encontrar buenas zonas antes que el grid), pero tampoco aprende.
- **Optimización bayesiana (TPE).** *Tree-structured Parzen Estimator* modela **P(hiperparámetros | buen score)** y **muestrea donde es probable mejorar**, usando el historial de pruebas. Encuentra buenas zonas con **muchas menos** evaluaciones.
- **Cuándo usar cada uno.** Espacio pequeño y barato de evaluar → Grid puede valer. Espacio grande / evaluación cara → bayesiana (Optuna) o Random. Muchísimos recursos en paralelo → Random/Hyperband escalan trivialmente.
- **Qué pasa si se omite el tuning inteligente.** Gastas presupuesto de cómputo en combinaciones malas y quizá no llegas al óptimo en el tiempo disponible.
- **Analogía.** Grid search es revisar TODOS los estantes del supermercado por orden. La optimización bayesiana es preguntar "¿dónde suele estar el café?" y caminar directo a esa zona, refinando con cada pista.

📄 FICHA DE CONCEPTO — Pruning (poda de pruebas) y Early stopping

- **Pruning.** Si tras los primeros folds una configuración va **claramente peor** que la mediana histórica, se **aborta** sin terminar los 5 folds. Redirige presupuesto desde configuraciones perdedoras hacia prometedoras (idea afín a *successive halving*). **n_warmup_steps** evita matar configuraciones que

arrancan lento.

- **Early stopping.** En los boosters, se añaden árboles **hasta que la métrica de validación deja de mejorar**, y entonces se para. Elige el número de árboles automáticamente y **controla la varianza** (evita seguir ajustando ruido).
- **Para qué sirven.** Acelerar el tuning (pruning) y **prevenir sobreajuste** eligiendo la complejidad justa (early stopping).
- **Cuidado clave.** El `eval_set` del early stopping debe transformarse con el preprocesador ajustado **solo en el fold-train**; si lo ajustas con datos de validación, **reintroduces leakage**. El proyecto lo hace explícitamente.
- **Analogía.** Pruning = no terminar de recorrer un pasillo del súper donde ya ves que no hay nada. Early stopping = dejar de estudiar un tema cuando tus simulacros dejan de mejorar, para no "sobre-empollar" detalles inútiles.

📄 FICHA DE CONCEPTO — Regularización (C en LogReg, L1/L2, λ)

- **Qué es.** Una penalización a la **complejidad** del modelo (al tamaño de sus coeficientes) que se añade a la función de pérdida. Cambia el equilibrio sesgo-varianza hacia menos varianza.
- **L2 (Ridge).** Penaliza la suma de **cuadrados** de los coeficientes. Los encoge hacia 0 (sin anularlos). Estabiliza ante multicolinealidad.
- **L1 (Lasso).** Penaliza la suma de **valores absolutos**. Puede llevar coeficientes **exactamente a 0** → selección de variables automática.
- **El parámetro C de la regresión logística.** Es la **inversa** de la fuerza de regularización: C grande = poca penalización (modelo más flexible, más varianza); C pequeño = mucha penalización (modelo más rígido, más sesgo). El tuning encontró $C \approx 14.86$ (regularización suave), coherente con un problema donde el modelo lineal no necesita encogerse mucho.
- **Cuándo usarla.** Casi siempre en modelos lineales (sobre datos escalados); imprescindible con muchas features o colinealidad.
- **Analogía.** La regularización es un "presupuesto de complejidad": obligas al modelo a gastar sus coeficientes con moderación, así no se inventa explicaciones rebuscadas para el ruido.

Resultado real (tras tuning, RSKF 5×3).

Modelo	F1(CV)	F1(train)	gap	mejores hiperparámetros
LogReg	0.6322	0.6340	+0.002	<code>C=14.86, penalty=l2</code>
HistGB	0.6320	0.7293	+0.097	<code>lr=0.027, max_leaf_nodes=49, min_samples_leaf=11, l2=5.39</code>
LightGBM	0.6288	0.7292	+0.100	<code>lr=0.014, num_leaves=54, min_child=72, subsample=0.89, colsample=0.61, λ=9.72</code>

El tuning **no cambia el veredicto**: LogReg sigue arriba y mantiene un gap casi nulo (+0.002), mientras los boosters, incluso regularizados, conservan un gap ~0.1 (más memorización). La optimización **confirma** la robustez del modelo simple en lugar de rescatar a los complejos.

Control del sobreajuste por tuning. Optimizar es, en sí, un riesgo: con suficientes intentos, alguna configuración "acierta" por azar en la CV (es el mismo fenómeno de las comparaciones múltiples, §10). Lo controlamos: (a) evaluando siempre por CV (nunca en test), (b) limitando los trials (40 c/u), (c) verificando el **gap train-CV** del modelo tuneado, y (d) confirmando con **nested CV** (§13), que es inmune a este sesgo.

📄 REGLA DE DECISIÓN — tuning con cabeza

- Espacio grande/caro → **Optuna (TPE) + pruning**. Pequeño → Grid puede bastar. Mucho paralelismo → Random/Hyperband.
- Boosters → **early stopping** con `eval_set` transformado solo con el fold-train.
- Tras tunear, **revisa el gap**: si crece, has sobreajustado el tuning → reduce trials o acota el espacio.
- Confirma la mejora con **nested CV** antes de crearla.

⚠ **ERROR TÍPICO — tunear mirando el test.** *Error:* usar el conjunto de test como criterio de selección de hiperparámetros. *Consecuencia:* el test deja de ser ciego; las métricas finales se inflan. El tuning vive **siempre** dentro de train+CV.

⚠ **ERROR TÍPICO — early stopping con leakage en el eval_set.** *Error:* ajustar el preprocesador con los datos de validación que luego usa el early stopping. *Consecuencia:* leakage sutil. El `eval_set` debe transformarse con el preprocesador del fold-train, no con el global.

Alternativas. `HalvingRandomSearchCV` (sklearn), Hyperband, *Bayesian Optimization* con procesos gaussianos (scikit-optimize), búsqueda aleatoria pura.

✅ QUÉ DEBES RECORDAR (tuning)

1. **Hiperparámetros** controlan la capacidad → el equilibrio sesgo-varianza; se eligen por **CV**, nunca por test.
2. **Optuna/TPE** encuentra buenas zonas con muchas menos evaluaciones que Grid; **pruning** acelera; **early stopping** regula la complejidad de los boosters.
3. **Regularización (C, L1/L2, λ)** es un presupuesto de complejidad; C es la **inversa** de la fuerza de penalización.
4. Tunear es un riesgo de sobreajuste en sí mismo: contróllalo con gap + **nested CV**.
5. Aquí el tuning **confirma** a LogReg (gap +0.002) en lugar de rescatar a los boosters (gap ~0.1).

12 · Diagnóstico de generalización (FASE 10)

```
learning_curve(pipe, X_train, y_train, cv=cv_skf, scoring="f1",
               train_sizes=np.linspace(0.1, 1.0, 6))
validation_curve(base_lr, ..., param_name="c1f__C", param_range=np.logspace(-3, 2, 8))
calibration_curve(y, proba, n_bins=10); roc_curve(...); precision_recall_curve(...)
```

Qué se hizo. Cinco diagnósticos visuales para confirmar que el modelo final ni sobre- ni infra-ajusta y que sus probabilidades son utilizables.

📄 FICHA DE CONCEPTO — Curva de aprendizaje (learning curve)

- **Qué es.** F1 de **train** y de **CV** en función del **tamaño de muestra** (10 %, 30 %, ..., 100 % del train).

- **Cómo leerla.** (a) Train alto y CV bajo, con **brecha grande que no se cierra** → **overfitting** (varianza): más datos ayudarían. (b) Train y CV **bajos y juntos** → **underfitting** (sesgo): más datos NO ayudan; necesitas más complejidad/mejores features. (c) Ambas **convergen y se aplanan** → estás cerca del límite; más datos del mismo tipo no moverán la aguja (caso del proyecto).
- **Para qué sirve.** Para decidir si la palanca es "**más datos**", "**más modelo**" o "**mejores features**".
- **Analogía.** Es la curva de progreso de un estudiante según cuántos ejercicios hace: si se estanca pronto, el problema no es la cantidad de práctica, sino el método o el material.

📄 FICHA DE CONCEPTO — Curva de validación (validation curve)

- **Qué es.** Métrica de train y CV en función de un **hiperparámetro** (aquí, **C** de LogReg en escala logarítmica).
- **Cómo leerla.** A la izquierda (C muy pequeño): ambos bajos → **sobre-regularizado** (underfitting). A la derecha (C muy grande): train sube y CV baja → **sobreajuste**. En medio: la **meseta óptima**. Sirve para verificar que el **C*** elegido por Optuna cae en esa meseta.
- **Para qué sirve.** Para *ver* el equilibrio sesgo-varianza de un hiperparámetro y confirmar que el tuning eligió bien.

📄 FICHA DE CONCEPTO — Calibración de probabilidades

- **Qué es.** Un modelo está **calibrado** si, entre los casos a los que asigna "0.7 de probabilidad", **aproximadamente el 70 % son realmente positivos**. La calibración mide si las probabilidades son *honestas*, no solo si el *ranking* es bueno.
- **Por qué importa.** Si el negocio usa la **probabilidad** (no solo la clase) para decidir presupuesto de retención, necesita confiar en ese número. Un modelo puede tener gran ROC-AUC (ordena bien) y estar **mal calibrado** (sus números no son tasas reales).
- **Cuidado con los pesos de clase.** `class_weight="balanced"` y `scale_pos_weight` **descalibran** las probabilidades (las inflan hacia la positiva). Si necesitas probabilidades calibradas, recalíbralas después (`CalibratedClassifierCV`).
- **Cómo se mide.** Curva de calibración (probabilidad predicha vs frecuencia observada por *bins*) y **Brier score**.
- **Analogía.** Es como el hombre del tiempo: si dice "70 % de lluvia" 10 veces, debería llover ~7 de esas. Un modelo bien calibrado permite presupuestar con sus porcentajes.

Lectura de ROC y PR aquí. (Las fichas de ROC-AUC/PR-AUC están en §9.) Se calculan a **todos los umbrales**; bajo desbalance, la **PR es más informativa** que la ROC, porque el eje FPR de la ROC se diluye con muchos negativos mientras que la precisión "siente" cada falso positivo.

⚠ **ERROR TÍPICO — diagnosticar sobre el test.** *Error:* trazar curvas de aprendizaje/calibración usando el test. *Consecuencia:* leakage. Estas curvas se calculan con probabilidades **out-of-fold** sobre **train**; el test sigue ciego.

⚠ **ERROR TÍPICO — fiarse de la probabilidad sin comprobar calibración.** *Error:* usar `predict_proba` como "probabilidad real" para decisiones de dinero sin verificar calibración. *Consecuencia:* presupuestos mal dimensionados. Una buena ROC-AUC **no** garantiza buena calibración.

Alternativas. Calibración posterior (Platt/Isotónica con `CalibratedClassifierCV`), curvas de *lift*/ganancia (muy usadas en marketing de retención), Brier score como métrica única de calibración + discriminación.

☑ QUÉ DEBES RECORDAR (diagnóstico)

1. La **curva de aprendizaje** dice si la palanca es más datos, más modelo o mejores features.
2. La **curva de validación** confirma que el hiperparámetro elegido cae en la meseta óptima (ni sobre- ni sub-regularizado).
3. **Calibración ≠ discriminación:** un modelo puede ordenar bien (ROC-AUC alta) y mentir en sus probabilidades.
4. Los pesos de clase **descalibran**; recalibra si el negocio usa la probabilidad para decidir.
5. Todos estos diagnósticos van sobre **train (out-of-fold)**, nunca sobre el test.

13 · Nested Cross-Validation (FASE 3)

```
outer = StratifiedKFold(5, ...)
for tr, va in outer.split(X_train, y_train):    # bucle externo = evaluación
    study.optimize(objective(X[tr], y[tr]), n_trials=20) # bucle interno = tuning
    pipe = build(study.best_params).fit(X[tr], y[tr])
    scores.append(f1_score(y[va], pipe.predict(X[va])))
```

Qué se hizo. Calculamos un estimador **insesgado** del error de generalización del *procedimiento completo* (incluido el tuning), aplicándolo a los dos finalistas (LogReg y HistGB).

📄 FICHA DE CONCEPTO — Nested Cross-Validation (validación cruzada anidada)

- **El problema que resuelve.** El F1 que reporta la CV de la §11 es **optimista**: los hiperparámetros se eligieron mirando *esa misma* CV, así que en parte mide la suerte de "el mejor de muchos sobre estos datos". Reportar ese número como rendimiento esperado sería sesgado al alza.
- **Cómo funciona.** Dos bucles. El **externo** (5 folds) sirve para **evaluar**. En cada fold externo, un bucle **interno** re-optimiza los hiperparámetros usando **solo** el train de ese fold; luego se evalúa en el val externo, que el tuning **nunca vio**. Tuning y evaluación usan datos **disjuntos**.
- **Para qué sirve.** Responde, sin sesgo, a la pregunta de negocio real: "*¿cuánto rendirá todo el pipeline (incluido el ajuste de hiperparámetros) sobre datos nuevos?*". Es el **estándar de oro** para estimar generalización cuando hay tuning.
- **Cuándo usarlo.** Siempre que tunees y quieras una cifra de generalización defendible, sobre todo en datasets pequeños/medianos donde el optimismo del tuning es notable.
- **Cuándo NO / con qué coste.** Es **caro** (tuning × folds externos). Con datasets enormes o presupuesto limitado, a veces se usa un único *validation set* separado (más barato, más ruidoso). Aquí se baja a 20 trials por fold para que sea asumible.
- **Qué pasa si se omite.** Reportas la cifra optimista de la CV de tuning como si fuera generalización: sobreestimas el rendimiento futuro.
- **Analogía.** Evaluar a un chef haciéndolo cocinar para 5 jurados distintos y, en cada caso, dejándole probar y ajustar solo con ingredientes que ese jurado **no** probará. Nadie califica un plato que el chef ya optimizó para su paladar.

Resultado real. `Nested CV LogReg: F1 = 0.6297 ± 0.0120 · Nested CV HistGB: F1 = 0.6296 ± 0.0137`. Las dos estimaciones insesgadas son **prácticamente idénticas** (diferencia 0.0001, muy por debajo de su desviación): la complejidad de HistGB **no aporta** generalización alguna sobre LogReg.

INTUICIÓN MATEMÁTICA — por qué 0.6297 ≈ 0.6296 cierra el caso. La diferencia entre ambos (0.0001) es **dos órdenes de magnitud** menor que su propia desviación (~0.012). En términos de señal/ruido, la diferencia es indistinguible de cero. Combinado con el Wilcoxon de §10, esto confirma —ya sin el sesgo optimista del tuning— el **empate técnico**: no hay ninguna base estadística para preferir el modelo complejo.

⚙️ REGLA DE DECISIÓN — ¿necesito nested CV?

- Tuneas hiperparámetros **y** necesitas reportar generalización → **sí**, nested CV (o un val set separado si no hay presupuesto).
- No tuneas (hiperparámetros por defecto) → una CV normal ya es (casi) insesgada; nested CV es opcional.

ERROR TÍPICO — reportar la CV de tuning como rendimiento esperado. *Error:* "mi mejor configuración da F1 = 0.6322 en CV, eso es lo que rendirá". *Consecuencia:* sobreestimación: ese número está sesgado al alza por haber elegido el mejor de muchos. La cifra honesta es la **nested** (0.6297), que coincide con el test (§16) → señal de que no hay leakage.

Alternativas. *Repeated nested CV* (más estable, más caro); un único *validation set* separado para tuning (más barato, más ruidoso).

✅ QUÉ DEBES RECORDAR (nested CV)

1. La CV usada para **tunear** da un score **optimista**; no es generalización.
2. **Nested CV** separa tuning (interno) y evaluación (externo) → cifra **insesgada**.
3. Aquí LogReg y HistGB dan nested F1 ≈ 0.630 idéntico → **empate técnico** confirmado sin sesgo.
4. Es el estándar de oro, pero **caro**; ajusta el presupuesto (menos trials por fold) sin perder el insesgamiento.

14 · Interpretabilidad (FASE 11)

```
coefs = lr_fit.named_steps["clf"].coef_[0]           # log-odds (signo + magnitud)
permutation_importance(lr_fit, X_test, y_test, scoring="f1") # impacto real, model-agnostic
shap.LinearExplainer(lr_fit.named_steps["clf"], Xtr_t)  # contribución por cliente
```

Qué se hizo. Tres lentes complementarias y, sobre todo, **interpretadas** (no solo dibujadas) para entender *por qué* predice el modelo.

📄 FICHA DE CONCEPTO — Coeficientes y log-odds (en regresión logística)

- **Qué es.** La regresión logística modela el **logaritmo de las probabilidades relativas** (log-odds) como una combinación lineal de las features:
 $\log[p/(1-p)] = \beta_0 + \beta_1 x_1 + \dots$. Cada coeficiente β_j es el cambio en log-odds por unidad de la feature j , **manteniendo las demás constantes**.
- **Cómo leerlo.** Signo: $\beta > 0$ empuja hacia churn; $\beta < 0$ protege. Magnitud: cuánto. Si exponencias el coeficiente, e^{β} es el **odds ratio**: cuántas veces se multiplican las probabilidades relativas por unidad de la feature.
- **Para qué sirve.** Interpretación global, transparente y barata: una de las grandes ventajas de elegir un modelo lineal.
- **Cuidado con la multicolinealidad (§3).** Con variables casi gemelas ($\text{total} \approx \text{monthly} \times \text{tenure}$), el crédito se **reparte** entre ellas y la magnitud individual de cada coeficiente debe leerse con cautela.
- **Analogía.** Es el "manual del modelo": una lista de qué sube y qué baja la probabilidad, y cuánto, todo lo demás igual.

📄 FICHA DE CONCEPTO — Permutation importance

- **Qué es.** Mide cuánto **empeora la métrica** (aquí F1) al **barajar** aleatoriamente una variable (destruyendo su relación con el target) mientras se dejan las demás intactas. Mucha caída → variable importante; nada de caída → irrelevante.
- **Por qué es buena.** Es **model-agnostic** (sirve para cualquier modelo) y mide la contribución a la **métrica real**, no a una heurística interna. Evita el sesgo de las importancias de árbol (`feature_importances_`), que **inflan** variables de alta cardinalidad.
- **Cuándo usarla.** Para una importancia global fiable y comparable entre modelos.
- **Cuidado con correlación.** Con variables correlacionadas, permutar una crea combinaciones **imposibles** (un cliente con tenure alto y total bajo), lo que puede distorsionar; además, si su "gemela" sigue presente, se **subestima** su efecto.
- **Analogía.** Es tapar un instrumento de la orquesta y oír cuánto se nota su ausencia. Si no cambia nada, no estaba aportando.

📄 FICHA DE CONCEPTO — SHAP (valores de Shapley)

- **Qué es.** Un método, basado en la **teoría de juegos**, que reparte la predicción de **cada caso individual** entre sus features de forma "justa". El valor SHAP de una feature para un cliente es su **aporte** a alejar la predicción de la media.
- **La propiedad clave (aditividad).** Para cada caso, la suma de los valores SHAP de todas las features **igual** exactamente la diferencia entre la predicción y la predicción media. Esto los hace **consistentes** y comparables entre clientes.
- **Para qué sirve.** Explicaciones **locales** (por cliente) y **globales** (agregando), accionables: permite retención personalizada.
- **Cuándo usarlo.** Cuando necesitas explicar **decisiones individuales** (y a menudo, justificarlas ante negocio o regulación). `LinearExplainer` para modelos lineales, `TreeExplainer` para árboles.
- **Cuidado.** SHAP explica **el modelo**, no **el mundo**: que el modelo use "fibra" no prueba que cambiar a DSL retenga al cliente (correlación ≠ causalidad, §4).
- **Analogía.** Es una **factura detallada** de por qué un cliente tiene 80 % de probabilidad de fuga: "+25 % por contrato mes a mes, +15 % por fibra, -10 % por 3 años de antigüedad...". Cada línea explica una parte del total.

Hallazgos convergentes (las tres lentes coinciden):

- **Empujan churn:** contrato *mes a mes*, *fibra óptica*, *cheque electrónico*, *paperless billing*, cargos mensuales altos.
- **Protegen:** mayor *tenure*, contratos de 1 y 2 años, *soporte técnico* y *seguridad online*, no tener internet.
- **Irrelevantes:** *gender*, *phone service* (coherente con su MI ≈ 0 de la auditoría, §3).

Por qué la convergencia importa. Que tres métodos de fundamentos **distintos** (paramétrico, por permutación, teoría de juegos) señalen los **mismos** drivers es la mejor garantía de que la explicación es robusta y no un artefacto de un método concreto. Si se contradijeran, desconfiaríamos.

🔑 REGLA DE DECISIÓN — qué método de interpretación usar

- Modelo **lineal** + interpretación global rápida → **coeficientes/log-odds**.
- Importancia global **fiable y comparable** entre modelos → **permutation importance** (mejor que `feature_importances_`).
- Explicar **casos individuales** / retención personalizada → **SHAP** (`TreeExplainer` para árboles, `LinearExplainer` para lineales).
- Con **fuerte correlación** entre features → prefiere métodos robustos a ella (ALE) y lee importancias con cautela.

⚠ **ERROR TÍPICO — confundir importancia con causalidad.** *Error:* "SHAP dice que la fibra empuja el churn → quitamos la fibra y retendremos". *Consecuencia:* decisión potencialmente inútil. SHAP/coeficientes explican **cómo predice el modelo**, no **qué causa la fuga**. Para causalidad: experimentos (A/B) o métodos causales.

⚠ **ERROR TÍPICO — fiarse de `feature_importances_` de árboles.** *Error:* rankear variables con la importancia interna de RF/boosting. *Consecuencia:* sesgo hacia variables de **alta cardinalidad** y hacia las que el árbol usó primero. La **permutation importance** (sobre la métrica) es más honesta.

Alternativas. *Partial Dependence/ICE* (efecto marginal de una variable), *LIME* (explicación local por aproximación lineal), *Accumulated Local Effects* (ALE, robusto a correlación entre features).

✅ QUÉ DEBES RECORDAR (interpretabilidad)

1. **Coeficientes/log-odds:** signo y magnitud del efecto de cada feature, todo lo demás constante (cuidado con colinealidad).
2. **Permutation importance:** caída de la métrica al barajar una variable; model-agnostic y sin el sesgo de `feature_importances_`.
3. **SHAP:** explica cada predicción individual con aportes que **suman** la diferencia respecto a la media (aditividad).
4. La **convergencia** de los tres métodos es la mejor señal de robustez de la explicación.
5. Interpretabilidad explica **el modelo**, no **el mundo**: nunca saltes de importancia a causalidad sin un experimento.

15 · Métricas, umbral y selección final (FASES 9 y 12)

```
# tabla comparativa final (default vs tuned): F1, MCC, PR-AUC, ROC-AUC, Recall, Precision,
# gap_train_cv, tiempos de entrenamiento e inferencia
# decisión programática: empate técnico (F1 >= top - tol) -> elegir el más simple/estable
# umbral óptimo por F1 con probabilidades out-of-fold (no se toca test)
prec, rec, thr = precision_recall_curve(y_train, oof); thr_opt = thr[argmax(F1)]
```

Qué se hizo. Construimos la tabla comparativa profesional (métricas + tiempos + estabilidad + ranking) para modelos *default* y *tuned*, fijamos la métrica primaria, optimizamos el **umbral de decisión** y aplicamos la regla de selección.

15.1 · Por qué F1/MCC y no Recall ni Accuracy

Esta es una de las decisiones más importantes del proyecto y donde el notebook original falló (criterio L3: recall puro).

- **Accuracy** premia acertar la clase mayoritaria → **engañosa** con desbalance (lo probó el baseline: 73 % "gratis").
- **Recall puro** se maximiza **prediciendo "todos churn"** (recall → 1, precisión → tasa base). Un negocio que llama a retención a **toda** su cartera no ahorra nada. Era el error degenerado del notebook original.
- **F1** equilibra precisión y recall (media armónica): castiga tanto perder fugas (FN) como molestar a clientes fieles (FP).
- **MCC** usa las cuatro casillas de la confusión: la métrica **más honesta** bajo desbalance (solo alta si aciertas en ambas clases).
- **PR-AUC** resume el rendimiento a todos los umbrales centrándose en la clase positiva.

🧠 **INTUICIÓN MATEMÁTICA — la trampa del recall puro.** Recall = $VP/(VP+FN)$. Si predices "churn" para **todos**, FN = 0 y Recall = 1.0 **automáticamente**, sin aprender nada — exactamente como el Dummy de mayoría daba Accuracy 0.73. Optimizar una métrica que tiene un **óptimo degenerado trivial** garantiza un "ganador" inútil. F1 y MCC no tienen ese atajo: para subirlos hay que acertar de verdad en ambas clases.

15.2 · Optimización del umbral de decisión

📄 FICHA DE CONCEPTO — Umbral de decisión y su optimización

- **Qué es.** Un clasificador produce una **probabilidad**; el **umbral** es el corte que la convierte en clase (por defecto 0.5: $p \geq 0.5 \rightarrow \text{churn}$). El umbral **no** cambia el modelo, solo **dónde** cortas.
- **Por qué 0.5 rara vez es óptimo con desbalance / pesos de clase.** Las probabilidades suelen estar descalibradas y la clase positiva es rara; mover el umbral reequilibra precisión y recall. Aquí el óptimo por F1 resultó **0.614**, no 0.5.
- **Cómo se optimiza sin hacer trampa.** Se barren todos los umbrales sobre probabilidades **out-of-fold** (de train, vía CV) y se elige el de máximo F1. **Nunca** se ajusta el umbral mirando el test (eso lo contaminaría).
- **Cuándo ajustarlo por otra cosa que no sea F1.** Si el negocio tiene **costes asimétricos** (un FN cuesta mucho más que un FP), el umbral debe salir de la **matriz de costes**, no de F1. Si quieres priorizar recall, usa **F-beta** (abajo).
- **Analogía.** Es la sensibilidad de una alarma de humo: muy sensible (umbral bajo) suena con el vapor de la ducha (FP); poco sensible (umbral alto) no avisa hasta que hay llamas (FN). El óptimo equilibra ambos según lo que te cueste cada error.

📄 FICHA DE CONCEPTO — F-beta (cuando recall y precisión no pesan igual)

- **Qué es.** Una generalización de F1: $F\beta = \frac{(1+\beta^2) \cdot P \cdot R}{(\beta^2 \cdot P + R)}$. $\beta = 1 \rightarrow F1$ (equilibrio). $\beta > 1 \rightarrow$ pondera más el **recall** ($\beta = 2$ es habitual cuando perder un positivo es caro). $\beta < 1 \rightarrow$ pondera más la **precisión**.
- **Cuándo usarla.** Cuando el coste de un FN y el de un FP son claramente distintos pero no quieres construir una matriz de costes completa.

- **Analogía.** Es el "mando de balance" entre cazar todos los positivos (recall) y no dar falsas alarmas (precisión).

15.3 · La regla de selección final

 **FICHA DE CONCEPTO — Navaja de Occam (parsimonia) en ML**

- **Qué es.** El principio de que, a **igualdad de capacidad explicativa**, debe preferirse la hipótesis más simple.
- **Su versión formal en ML.** A **igualdad de error de validación**, el modelo de **menor complejidad** tiene **menor varianza** y, por tanto, mayor probabilidad de mantener su rendimiento en datos nuevos (y de no romperse ante pequeños cambios de la población). Simplicidad = robustez, no solo elegancia.
- **Por qué aquí es decisiva.** El top de modelos es estadísticamente **indistinguible** (§10, Wilcoxon $p > 0.05$) y su **nested CV** coincide (§13). Entre iguales, se elige el **más simple, estable, interpretable y barato**: la **Regresión Logística**.
- **Beneficios concretos del simple aquí.** Menor gap (no memoriza), coeficientes interpretables (negocio), inferencia instantánea, sin dependencias pesadas, fácil de mantener y auditar.
- **Analogía.** Si dos rutas te llevan al mismo sitio en el mismo tiempo, eliges la que tiene menos cruces donde algo pueda salir mal.

Decisión final (REGLA FINAL aplicada). No se elige LogReg por tener el F1 más alto por milésimas, sino porque, **entre modelos estadísticamente equivalentes**, es el de **menor riesgo de sobreajuste y máxima robustez** — exactamente lo que pide la jerarquía del proyecto (§Filosofía). Se acompaña de su **umbral calibrado** (0.614).

 **REGLA DE DECISIÓN — elegir entre modelos que empatan**

- Empate estadístico ($p > 0.05$, IC solapados, nested CV $\approx$) $\rightarrow$ elige por: **menor gap > interpretabilidad > coste de inferencia/mantenimiento > menos dependencias**. El rendimiento ya está empatado; decide la robustez.
- Optimiza el umbral por **F1 out-of-fold** (o por coste de negocio si lo hay), **nunca** sobre el test.

 **ERROR TÍPICO — optimizar recall (o accuracy) en solitario.** *Error:* `refit="recall"` en la búsqueda de hiperparámetros (defecto L3 del notebook original). *Consecuencia:* el "ganador" tiende a predecir casi todo como positivo; recall ≈ 1 , precisión $\approx$ tasa base, modelo inútil.

 **ERROR TÍPICO — dejar el umbral en 0.5 por inercia.** *Error:* usar el corte por defecto con clases desbalanceadas y pesos de clase. *Consecuencia:* dejas rendimiento sobre la mesa (aquí, 0.614 mejora F1, MCC, precisión y accuracy frente a 0.5).

 **ERROR TÍPICO — optimizar el umbral sobre el test.** *Error:* elegir el umbral que maximiza F1 en el conjunto de test. *Consecuencia:* contaminas el test; su métrica deja de ser honesta. El umbral sale de **out-of-fold** de train.

Alternativas. Umbral por máximo **MCC**; por **F-beta** ($\beta > 1$ prioriza recall); por **mínimo coste esperado** dada una matriz de costes de negocio (lo más correcto cuando los costes FN/FP están cuantificados).

 **QUÉ DEBES RECORDAR (selección)**

1. **Accuracy y recall puro engañan** con desbalance; usa **F1/MCC/PR-AUC** como métricas de cabecera.
2. El **umbral** es una palanca aparte del modelo: optimízalo **out-of-fold**, nunca sobre el test; 0.5 rara vez es óptimo.
3. **F-beta** o una **matriz de costes** cuando FN y FP no pesan igual.
4. **Navaja de Occam:** entre modelos que empatan estadísticamente, gana el más simple/estable/interpretable (menor varianza = mejor generalización).
5. La elección final fue LogReg + umbral 0.614 por **robustez**, no por décimas de F1.

16 · Evaluación final en el test aislado

```
win_pipe.fit(X_train, y_train) # entrena en todo el train
proba_test = win_pipe.predict_proba(X_test)[:,-1] # PRIMER contacto con el test
pred_opt = (proba_test >= thr_opt).astype(int)
metricas_completas(y_test, pred_opt, proba_test)
```

Qué se hizo. El **único** contacto con el test. Entrenamos el pipeline ganador (LogReg tuneado, **C=14.86**) en **todo** el train, aplicamos el umbral elegido (**0.614**) y medimos. Estas son **las métricas honestas de generalización**: las que el cliente puede esperar en producción.

Resultado real en el test aislado (n = 1407).

Métrica	umbral 0.5	umbral 0.614
Accuracy	0.7441	0.7790
Balanced Acc	0.7609	0.7514
Precision	0.5120	0.5692
Recall	0.7968	0.6925
F1	0.6234	0.6248
MCC	0.4681	0.4748
ROC-AUC	0.8446	0.8446
PR-AUC	0.6517	0.6517

`classification_report` (umbral 0.614): clase **Evade** $\rightarrow$ precision 0.57, recall 0.69, F1 0.62 (soporte 374); clase **Permanece** $\rightarrow$ precision 0.88, recall 0.81, F1 0.84 (soporte 1033).

Coherencia total. F1 test = **0.625** $\approx$ F1 CV (0.632) $\approx$ F1 nested CV (0.630). Que las tres cifras coincidan (sin que el test sorprenda a la baja) es la **prueba empírica de que no hay leakage** y de que el modelo generaliza. El umbral 0.614 sube accuracy, precisión, F1 y MCC a costa de algo de recall — un intercambio ajustable según el coste de negocio.

📄 FICHA DE CONCEPTO — Estimador insesgado y la coherencia CV $\approx$ test

- **Qué es un estimador insesgado.** Una forma de medir cuyo valor esperado coincide con la cantidad real que quieres estimar (aquí, el rendimiento poblacional). El test, **aislado desde \$6 y nunca usado para decidir**, es una muestra i.i.d. de la población objetivo; su métrica es un estimador insesgado del rendimiento futuro (con su propio error de muestreo, acotable por $n = 1407$).
- **Por qué la coincidencia CV $\approx$ nested $\approx$ test "cierra el círculo".** Si hubiera leakage, el test (el único dato verdaderamente ciego) caería **por debajo** de la CV. Que coincidan es la confirmación empírica de que el protocolo es limpio.
- **Cuándo desconfiar.** Test **mucho peor** que CV $\rightarrow$ leakage en el protocolo o sobreajuste al proceso. Test **mucho mejor** que CV $\rightarrow$ el test se contaminó (alguien lo miró) o el split no es representativo.
- **Analogía.** Es el examen final a libro cerrado: si tu nota se parece a la de los simulacros (CV), estudiaste de verdad; si es mucho peor, hacías trampa en los simulacros (leakage).

⚠ **ERROR TÍPICO — "ajustar una cosita más" después de ver el test.** *Error:* tras mirar el test, cambiar el umbral, el modelo o las features "para que dé mejor". *Consecuencia:* el test **se quema**: cualquier decisión posterior basada en él lo convierte en validación, y su métrica deja de ser honesta. Si necesitas reoptimizar, necesitas datos nuevos.

⚠ **ERROR TÍPICO — reportar el test como una única cifra sin incertidumbre.** *Error:* "F1 de test = 0.625" a secas. *Consecuencia:* un solo test tiene varianza. Idealmente se acompaña de un **IC por bootstrap** sobre el test para comunicar el margen de error.

Alternativas. Bootstrap del test para IC de las métricas; validación temporal *out-of-time* si hubiera fechas; *backtesting* en producción con A/B test.

☑ QUÉ DEBES RECORDAR (test)

1. El test se toca **una vez**, al final, con el pipeline ya decidido. Es la métrica honesta de generalización.
2. **CV $\approx$ nested CV $\approx$ test** es la firma empírica de **ausencia de leakage**.
3. Tras ver el test, **no ajustes nada**: lo contaminarías.
4. Una sola cifra de test tiene varianza; acompáñala de un IC (bootstrap) si puedes.

17 · Exportación del modelo (MLOps)

```
artefacto = {"pipeline": win_pipe, "threshold": thr_opt, "features_input": list(X.columns),
            "target": TARGET, "metricas_test": {...}, "modelo": win_name,
            "sklearn_version": sklearn.__version__, "random_state": RANDOM_STATE}
joblib.dump(artefacto, "mejores_practicas/modelo_final.pkl")
```

Qué se hizo. Guardamos el **pipeline completo** (preprocesador + modelo en un solo objeto) más el umbral y los metadatos. En producción, scorear es `pipeline.predict_proba(nuevos_datos)` y comparar con `threshold`: **sin pasos manuales** que puedan reintroducir leakage o divergencia.

📄 FICHA DE CONCEPTO — Training-serving skew y por qué empaquetar el pipeline

- **Qué es.** La divergencia entre el preprocesamiento de **entrenamiento** y el de **producción** (*servicing*). Si en entrenamiento estandarizaste con ciertas medias y en producción reimplementas el cálculo a mano (con otras medias, otro orden de columnas, otra gestión de categorías nuevas), el modelo recibe entradas distintas a las que aprendió.
- **Por qué es una causa típica de degradación silenciosa.** No lanza errores: simplemente, las predicciones empeoran sin que nadie sepa por qué.
- **Cómo lo elimina el pipeline.** Empaquetar preprocesador + modelo en un único **Pipeline** garantiza que **las mismas medias, escalas y categorías** aprendidas en train se apliquen en inferencia, por construcción.
- **Analogía.** Es entregar una máquina de café ya programada en lugar de una bolsa de granos con instrucciones: el receptor obtiene exactamente el mismo café sin posibilidad de equivocarse en la receta.

📄 FICHA DE CONCEPTO — Data drift (deriva de datos)

- **Qué es.** Que la distribución de los datos en producción **cambie** respecto a la de entrenamiento (nuevos hábitos, nuevos productos, estacionalidad, cambios de mercado). El modelo asume que el futuro se parece al pasado; si deja de ser cierto, se degrada.
- **Tipos.** *Covariate shift* (cambian las features), *label shift* (cambia la proporción de clases), *concept drift* (cambia la **relación** entre features y target — lo más peligroso).
- **Qué hacer. Monitorizar** la distribución de entrada y las métricas sobre etiquetas reales cuando lleguen; reentrenar periódicamente; alertar ante cambios bruscos.
- **Analogía.** Un mapa de carreteras envejece: si construyen autopistas nuevas (drift), tu ruta óptima de hace dos años ya no lo es. Hay que actualizar el mapa (reentrenar).

🔑 REGLA DE DECISIÓN — empaquetado y despliegue

- Guarda **siempre** el pipeline **completo** (no el modelo "desnudo") + umbral + metadatos (versión de librerías, semilla, features esperadas, métricas de test).
- Empaqueta para evitar *skew*; **monitoriza** drift en producción; planifica reentrenamientos.
- Pickle solo para **artefactos de confianza**; para portabilidad o seguridad, evalúa ONNX/*skops*.

⚠ **ERROR TÍPICO — reimplementar el preprocesamiento en producción.** *Error:* guardar solo el estimador y recodificar/reescalar a mano en el servicio. *Consecuencia:* *training-serving skew* y degradación silenciosa. Empaqueta el pipeline entero.

 **ERROR TÍPICO — cargar un pickle sin precaución / sin versión.** *Error:* `joblib.load` de un artefacto de origen no confiable, o sin registrar la versión de sklearn. *Consecuencia:* (a) un pickle **ejecuta código** al cargarse → riesgo de seguridad; (b) incompatibilidad de versiones que rompe la carga. Carga solo artefactos de confianza y guarda la versión.

Alternativas. ONNX/PMML (portabilidad entre lenguajes), `skops` (serialización más segura que pickle), registro de modelos (MLflow) con versionado, métricas y *lineage*.

 **QUÉ DEBES RECORDAR (MLOps)**

1. Despliega el **pipeline completo + umbral + metadatos**, no el modelo suelto.

2. El **training-serving skew** es una causa silenciosa de degradación; el pipeline lo elimina por construcción.

3. Vigila el **data drift** (especialmente el *concept drift*) y planifica reentrenamientos.

4. El **pickle ejecuta código** al cargarse: solo artefactos de confianza; guarda la versión de las librerías.

5. El umbral se ajusta al coste de negocio **sin reentrenar** (es un parámetro de decisión, no del modelo).

18 · Conclusiones

1. **Generalización por encima de la métrica.** LogReg, HistGB, LightGBM y CatBoost son **estadísticamente indistinguibles** en F1 (Wilcoxon $p > 0.05$) y su nested CV coincide. La **REGLA FINAL** ordena elegir el simple: **Regresión Logística** con umbral calibrado — menor varianza, menor riesgo de overfitting, interpretable y barata.
2. **Cero data leakage.** Todo el preprocesamiento vive en pipelines; se ajusta solo con el fold-train; el test permaneció ciego hasta \$16; el umbral salió de probabilidades out-of-fold. Se corrigen L1 (scaler sobre todo el dataset) y L2 (selección de features mirando test) del notebook original.
3. **Robustez estadística.** RSKF 5×3 + IC 95 % + Wilcoxon pareado + nested CV: ningún ganador por diferencias marginales.
4. **Interpretabilidad triangulada.** Coeficientes, permutation importance y SHAP coinciden en los drivers: contrato mes a mes, fibra, baja antigüedad y cheque electrónico empujan churn; contratos largos, antigüedad y soporte protegen.
5. **Límite teórico reconocido.** El ruido irreducible (perfiles idénticos con churn opuesto) fija un techo de Bayes; un F1 honesto ~0.62–0.64 es coherente, no una limitación del modelo.

Tabla: notebook original vs. reconstrucción

Dimensión	Notebook original	mejores_practicas
Preprocesamiento	<code>fit_transform</code> sobre todo X (leakage L1)	Pipelines cerrados, fit por fold
Selección de features	mirando el test (leakage L2)	por CV / sin selección sesgada
Métrica de selección	Recall puro (degenerada)	F1 / PR-AUC / MCC + negocio
Modelos	Dummy, RF, XGB	9 familias + 3 tuneadas
Tuning	GridSearch exhaustivo	Optuna (TPE) + pruning + early stopping
Comparación	1 partición, sin incertidumbre	RSKF + IC95 + Wilcoxon + nested CV
Variable redundante	<code>charges_day</code> incluida	eliminada (probado = monthly/30)
Interpretabilidad	<code>feature_importances_</code>	coef + permutation + SHAP
Reproducibilidad	<code>device='cuda'</code> fijo, celdas frágiles	semilla global, CPU, pipeline serializado
Modelo final	Random Forest (por recall)	Regresión Logística (por generalización)

Recomendaciones de negocio

- **Retención dirigida** a clientes mes-a-mes con baja antigüedad y fibra: máximo riesgo y máximo retorno de la acción.

• **Incentivar contratos de 1–2 años y pagos automáticos:** son los factores protectores más fuertes y accionables.

• **Revisar la propuesta de valor de la fibra óptica:** su tasa de fuga (41.9 %) sugiere un problema de precio/calidad percibida.

• **Usar la probabilidad calibrada**, no solo la clase, y fijar el umbral según el coste de un falso negativo (cliente perdido) frente a un falso positivo (incentivo regalado).

 **QUÉ DEBES RECORDAR (conclusiones)**

1. El proyecto demuestra, con evidencia, que **la complejidad no compró rendimiento**: ganó el modelo simple por robustez.

2. La **coherencia CV ≈ nested ≈ test** certifica que las métricas son honestas (sin leakage).

3. Las decisiones se tomaron por **evidencia y estadística**, no por convención ni por la métrica más vistosa.

4. La interpretabilidad **triangulada** convierte el modelo en acciones de negocio concretas y defendibles.

19 · Reglas de decisión consolidadas

Este capítulo reúne, en un solo lugar, las heurísticas dispersas por el documento. Son **reglas por defecto**, no dogmas: indican el camino que acierta la mayoría de las veces y, cuando procede, la excepción. Léelas como "si ves X, tu primera hipótesis de acción es Y; confírmala con evidencia".

19.1 · Datos y variables

Si ocurre X...	...normalmente haz Y	Por qué / matiz
Una columna tiene tantos valores únicos como filas	Elimínala (es un identificador)	Señal cero, riesgo de memorización. Excepción: si codifica orden/tiempo, extrae esa señal

Si ocurre X...	...normalmente haz Y	Por qué / matiz
Catógorica nominal de baja cardinalidad	One-Hot Encoding	No inventa orden; un coeficiente por categoría
Catógorica con orden real ($S < M < L$)	Ordinal Encoding	El número codifica información verdadera
Catógorica de alta cardinalidad (cientos)	Target/Frequency encoding (con CV interna)	One-Hot explotaría en columnas dispersas
Dos columnas con correlación ≈ 1 (redundancia exacta)	Elimina una	No aporta info y reparte el crédito (colinealidad)
Colinealidad fuerte pero no exacta	Conserva, pero documenta; cuidado al interpretar; $VIF > 5-10 \rightarrow$ reduce	Daña la interpretación lineal, no tanto la predicción de árboles
Variable con $MI \approx 0$	No la elimines automáticamente	Puede tener señal en interacción; elimina solo redundancias probadas
Hay valores faltantes	Imputa dentro del pipeline	Imputar antes del split = leakage
Quieres crear una feature nueva	Acéptala solo si mejora la CV (no el train)	"Podría ayudar" no basta; debe demostrarlo

19.2 · Validación y leakage

Si ocurre X...	...normalmente haz Y	Por qué / matiz
Empiezas cualquier proyecto	Aísla el test antes de mirar nada	Cualquier decisión mirando el test lo contamina
Clase objetivo desbalanceada	Estratifica el split y los folds	Mantiene la proporción; estabiliza métricas
Dataset pequeño/mediano	Validación cruzada (no hold-out único)	Un solo hold-out es ruidoso
Necesitas comparar modelos con poco ruido	Repeated Stratified K-Fold	Más estimaciones $\rightarrow$ SEM menor ($\sqrt{k}$)
Tuneas hiperparámetros y reportas generalización	Nested CV	La CV de tuning es optimista
Hay tiempo en los datos	TimeSeriesSplit (no aleatorio)	No mezclar futuro con pasado
Hay grupos (mismo cliente varias filas)	GroupKFold	El mismo grupo no puede estar en train y test
Cualquier preprocesamiento con parámetros aprendidos	Mételo en un Pipeline	Se reajusta por fold $\rightarrow$ sin leakage
Sospechas leakage	Encapsula todo en pipelines; compara CV vs test	Más folds NO arreglan leakage (reducen varianza, no sesgo)
Test $\gg$ CV o test $\ll$ CV	Investiga: contaminación o leakage	Coherencia CV $\approx$ test = protocolo limpio

19.3 · Modelos y preprocesamiento

Si ocurre X...	...normalmente haz Y	Por qué / matiz
Modelo basado en distancia/kernel/lineal/red	Escala las numéricas	Las distancias/penalizaciones asumen escalas comparables
Modelo basado en árboles/boosting	No escales	Parten por umbrales; escalar es inútil
Hay outliers y vas a escalar	RobustScaler (o PowerTransformer si hay sesgo)	Mediana/IQR resisten extremos; MinMax no
Sin outliers, quieres interpretabilidad	StandardScaler	El más estándar e interpretable
Clase desbalanceada	Pesos de clase primero (luego umbral/métricas robustas)	No inventa datos como SMOTE
El patrón parece casi lineal	Prueba un modelo lineal en serio	Menor varianza; puede ganar (¡aquí lo hizo!)
$F1_{train} \approx 1.0$ y CV mediocre	El modelo memoriza $\rightarrow$ simplifica/regulariza	Gap grande = overfitting
Train $\approx$ CV pero ambos bajos	Underfitting $\rightarrow$ más complejidad/mejores features	Alto sesgo, no varianza
Espacio de hiperparámetros grande/caro	Optuna (TPE) + pruning	Encuentra buenas zonas con menos evaluaciones

19.4 · Métricas y selección

Si ocurre X...	...normalmente haz Y	Por qué / matiz
Clasificación desbalanceada	Métrica primaria F1/MCC/PR-AUC , nunca Accuracy	Accuracy premia la clase mayoritaria
Quieres comparar capacidad de ranking	ROC-AUC (y PR-AUC si hay desbalance)	Independientes del umbral
Un falso negativo es muy caro	Sube recall: F-beta ($\beta > 1$) o baja el umbral	Coste asimétrico
Un falso positivo es muy caro	Sube precisión: F-beta ($\beta < 1$) o sube el umbral	Coste asimétrico

Si ocurre X...	...normalmente haz Y	Por qué / matiz
Tienes costes FN/FP cuantificados	Umbral por mínimo coste esperado (matriz de costes)	Más correcto que F1 genérico
Clases desbalanceadas + pesos de clase	Optimiza el umbral (no dejes 0.5) out-of-fold	0.5 rara vez es óptimo; nunca sobre el test
Dos modelos difieren por milésimas	Test estadístico pareado (Wilcoxon) antes de decidir	Puede ser ruido de muestreo
Modelos empatan estadísticamente	Elige el más simple/estable/interpretable	Navaja de Occam = menor varianza
Necesitas confiar en la probabilidad	Verifica calibración (y recalibra si usas pesos)	ROC-AUC alta ≠ probabilidades honestas

19.5 · Interpretabilidad y despliegue

Si ocurre X...	...normalmente haz Y	Por qué / matiz
Quieres importancia global fiable	Permutation importance	Sin el sesgo de <code>feature_importances_</code>
Necesitas explicar casos individuales	SHAP	Aporte por feature, suma = predicción – media
Tentado de saltar de importancia a causa	No lo hagas sin experimento (A/B)	Explica el modelo, no el mundo
Vas a producción	Empaqueta pipeline + umbral + metadatos	Evita training–serving skew
Modelo en producción	Monitoriza data drift ; reentrena periódicamente	El concept drift degrada en silencio
Cargas un <code>.pkl</code>	Solo de confianza ; guarda versión de librerías	Pickle ejecuta código al cargar

☒ **QUÉ DEBES RECORDAR** (reglas de decisión)

1. Son **puntos de partida** con alta tasa de acierto, no leyes: confirma siempre con evidencia (CV, test estadístico).
2. La columna "por qué" importa tanto como la regla: entender el motivo te deja **reconocer la excepción**.
3. Casi todas reducen a tres ideas: **no filtres información** (leakage), **no confundas ruido con señal** (estadística) y **no pagues complejidad que no compra generalización** (Occam).

20 · Catálogo de métricas: clasificación vs regresión

Una de las confusiones más frecuentes —y que más cuesta en entrevistas y en proyectos reales— es **usar una métrica que no corresponde al tipo de problema**. La regla de oro es simple pero absoluta:

La métrica la dicta la naturaleza de la variable objetivo, no tus ganas.

- Si el target es una **categoría** (sí/no, clase A/B/C) → es **clasificación** → métricas de **acierto/error de etiqueta** (Accuracy, Precision, Recall, F1, MCC, ROC-AUC...).
- Si el target es un **número continuo** (precio, temperatura, demanda) → es **regresión** → métricas de **distancia entre números** (MAE, RMSE, R²...).

Por qué no se pueden mezclar (la intuición de fondo). Recall, F1 o Accuracy se calculan a partir de la **matriz de confusión**, que cuenta aciertos y errores de **etiqueta** (verdaderos/falsos positivos/negativos). En regresión **no existe "acierto" ni "fallo"**: predecir 99.8 cuando el valor real es 100 no es ni un acierto ni un error binario, es un error de **0.2 unidades**. No hay verdaderos positivos que contar, así que F1/Recall/Accuracy **no están definidos**. A la inversa, R² o RMSE miden distancias entre números reales; aplicarlos a etiquetas "0/1" daría un número, pero **carente de la interpretación** que buscas (no te dice nada sobre detección de la clase rara).

Nota sobre "estático" vs series de tiempo. Este capítulo cubre clasificación y regresión **estáticas** (cada fila es independiente, sin orden temporal). Las **series de tiempo** son una familia aparte: añaden métricas y validaciones propias (MASE, sMAPE, validación con `TimeSeriesSplit/walk-forward`, respeto estricto del orden temporal para no usar el futuro). La diferencia clave: en problemas estáticos puedes barajar las filas; en series de tiempo, **jamás** (barajar destruiría la dependencia temporal y produciría leakage del futuro). Las métricas de error puntual (MAE, RMSE) se siguen usando en forecasting, pero acompañadas de las específicas y de un protocolo de validación temporal.

20.1 · Métricas de CLASIFICACIÓN (target categórico)

Todas se derivan de la **matriz de confusión** (VP, VN, FP, FN) o de las **probabilidades** predichas.

Métrica	Qué mide / fórmula	Rango (mejor)	Cuándo usarla	Cuándo NO	Riesgo / error típico
Accuracy	Aciertos totales $(VP+VN)/N$	0–1 (1)	Clases equilibradas y costes simétricos	Desbalance (la infla la clase mayoritaria)	Reportarla con 90/10 y creer que el modelo es bueno
Balanced Accuracy	Media de recall por clase	0–1 (1)	Desbalance, si quieres tratar ambas clases por igual	Si una clase importa mucho más que la otra	Confundirla con Accuracy normal
Precision	De los predichos positivos, ¿cuántos lo son? $VP/(VP+FP)$	0–1 (1)	Cuando un FP es caro (alarma molesta, intervención costosa)	Si ignorar positivos reales es lo grave	Optimizarla sola → predecir positivo solo en los casos obvios
Recall / Sensibilidad	De los positivos reales, ¿cuántos detecté? $VP/(VP+FN)$	0–1 (1)	Cuando un FN es caro (cáncer, fuga, fraude)	Sola : óptimo degenerado = predecir todo positivo	Maximizarla en solitario → modelo inútil (defecto L3)

Métrica	Qué mide / fórmula	Rango (mejor)	Cuándo usarla	Cuándo NO	Riesgo / error típico
Specificity	De los negativos reales, ¿cuántos acerté? $VN/(VN+FP)$	0–1 (1)	Cuando importa no marcar negativos como positivos	Como única métrica con desbalance	Olvidarla al centrarse solo en recall
F1-score	Media armónica de P y R $2PR/(P+R)$	0–1 (1)	Desbalance + ambos errores importan	Si los costes FN/FP son muy asimétricos (usa F-beta)	Crear que F1 alto implica buena calibración
F-beta	F1 ponderada $(1+\beta^2)PR/(\beta^2P+R)$	0–1 (1)	Costes asimétricos sin matriz de costes ($\beta > 1$: recall)	Si tienes la matriz de costes real (úsala)	Elegir β arbitrariamente sin justificar el negocio
MCC	Correlación real–predicho (4 casillas)	–1 a +1 (1)	La más honesta con desbalance	Si necesitas algo interpretable por negocio directamente	Ignorarla por ser "menos famosa" que F1
Cohen's Kappa	Acuerdo corregido por azar	–1 a +1 (1)	Acuerdo entre clasificadores/anotadores	Como métrica de negocio principal	Interpretar su escala sin contexto
ROC-AUC	Área ROC; calidad del ranking	0.5–1 (1)	Comparar capacidad de ordenación; costes/umbral por decidir	Desbalance fuerte (se ve optimista) → usa PR-AUC	Reportarla con 99/1 y creer que basta
PR-AUC (AP)	Área Precision-Recall; foco en positiva	tasa base–1 (1)	Desbalance + interesa la clase rara	Si las clases están equilibradas (ROC vale)	Comparar PR-AUC entre datasets con distinta tasa base
Log-loss	Penaliza probabilidades mal calibradas	0–∞ (1)	Cuando importan probabilidades bien calibradas	Si solo te importa la etiqueta final	Castiga durísimo una predicción confiada y equivocada
Brier score	Error cuadrático de la probabilidad	0–1 (1)	Medir calibración + discriminación juntas	Como única métrica de discriminación	Confundir buena Brier con buen recall

Cómo elegir la métrica primaria en clasificación (árbol de decisión):

- 1. ¿Las clases están **equilibradas** y los errores cuestan **lo mismo**? → **Accuracy** es razonable.
- 2. ¿Hay **desbalance**? → olvida Accuracy. ¿Importan **ambas** clases? → **MCC** o **F1**. ¿Solo la positiva? → **PR-AUC**.
- 3. ¿El **coste FN** ≠ **coste FP**? → **F-beta** (o, mejor, **umbral por matriz de costes**).
- 4. ¿Vas a usar la **probabilidad** (no solo la clase)? → vigila **Log-loss** / **Brier** y la **calibración**.
- 5. ¿Solo necesitas **ordenar** (priorizar) sin fijar umbral? → **ROC-AUC** (PR-AUC si hay desbalance).

📖 **INTUICIÓN MATEMÁTICA — por qué media armónica en F1.** La media armónica $2PR/(P+R)$ está **siempre por debajo** de la aritmética y se acerca al **menor** de los dos valores. Si $P = 0.9$ y $R = 0.1$: aritmética = 0.5, pero armónica ≈ 0.18. Así, F1 solo es alto si **ambos** lo son: castiga el desequilibrio, justo lo que queremos para que nadie "haga trampa" inflando una a costa de la otra.

⚠️ **ERROR TÍPICO (entrevista) — usar Accuracy con 99/1.** Error: "mi modelo de fraude tiene 99 % de accuracy". Consecuencia: probablemente predice "no fraude" siempre (el 99 % es la tasa base). La respuesta correcta menciona **PR-AUC, Recall y MCC**, no Accuracy.

20.2 · Métricas de REGRESIÓN (target numérico continuo)

Aquí **no hay matriz de confusión**: se mide la **distancia** entre el número predicho $\hat{y}$ y el real y (el **residuo** $e = y - \hat{y}$).

📖 **FICHA DE CONCEPTO — Residuos**

- **Qué son.** La diferencia entre lo real y lo predicho: $e_i = y_i - \hat{y}_i$. Son la materia prima de toda métrica de regresión y del diagnóstico del modelo.
- **Para qué sirven.** Analizar su **patrón** revela problemas: si los residuos crecen con $\hat{y}$ (forma de embudo) hay **heterocedasticidad**; si tienen estructura (curva), el modelo no captó una relación; si tienen sesgo (media ≠ 0), el modelo subestima o sobreestima sistemáticamente.
- **Qué buscar (en regresión lineal).** Residuos centrados en 0, sin patrón, de **varianza constante** (homocedasticidad) y aproximadamente normales — son los supuestos que hacen fiables los intervalos y los p-valores.

📖 **FICHA DE CONCEPTO — Homocedasticidad (y heterocedasticidad)**

- **Homocedasticidad.** La varianza de los residuos es **constante** a lo largo del rango de predicción. Es un supuesto clave de la regresión lineal clásica.
- **Heterocedasticidad.** La varianza **cambia** (p. ej. errores mayores para valores grandes). No sesga las predicciones, pero **invalida los errores estándar** de los coeficientes (los intervalos de confianza y p-valores dejan de ser fiables).
- **Qué hacer si aparece.** Transformar el target (log), usar errores estándar robustos, o modelos que no asuman varianza constante.
- **Analogía.** Es como una báscula cuyo margen de error crece con el peso: sirve para pesar plumas, pero no te fíes de su precisión con elefantes. La medida media puede ser correcta; su *incertidumbre* no es la que crees.

Métrica	Qué mide / fórmula	Rango (mejor)	Cuándo usarla	Cuándo NO	Riesgo / error típico
MAE	Error absoluto medio $\text{mean}(y-\hat{y})$	0–∞ (1)	Quieres error en unidades reales , robusto a outliers	Si los errores grandes deben penalizar más	Olvidar que trata todos los errores por igual
MSE	Error cuadrático medio $\text{mean}((y-\hat{y})^2)$	0–∞ (1)	Penalizar fuerte los errores grandes; base matemática	Interpretabilidad (está en unidades ²)	Comparar MSE entre datasets de distinta escala

Métrica	Qué mide / fórmula	Rango (mejor)	Cuándo usarla	Cuándo NO	Riesgo / error típico
RMSE	Raíz del MSE	0–∞ (↓)	Como MSE pero en unidades reales ; muy usada	Cuando hay outliers que no quieres que dominen	Sensible a outliers (los eleva al cuadrado)
R²	Varianza explicada $1 - \frac{SS_{res}}{SS_{tot}}$	≤1 (↑)	Comunicar "% de variabilidad explicada"	Comparar entre datasets distintos; puede ser negativo	Crear que R² alto = buen modelo (puede haber overfit)
R² ajustado	R² penalizado por nº de variables	≤1 (↑)	Comparar modelos con distinto nº de features	Si no comparas complejidad	Usar R² normal y "premiar" añadir variables inútiles
MAPE	Error porcentual absoluto medio	0–∞% (↓)	Error relativo , comparar escalas distintas	Si hay valores reales cerca de 0 (explota)	División por ~0; penaliza asimétricamente
MedAE	Mediana del error absoluto	0–∞ (↓)	Muy robusta a outliers	Si los outliers son justo lo que importa	Ocultar errores grandes en la cola
RMSLE	RMSE del logaritmo	0–∞ (↓)	Target con crecimiento exponencial ; penaliza subestimar	Valores negativos o cero sin tratar	Olvidar que penaliza más la subestimación
Huber loss	Híbrido MAE/MSE	0–∞ (↓)	Robustez a outliers y sensibilidad a errores medios	Si necesitas una métrica estándar y simple	Elegir mal el umbral δ

Cómo elegir la métrica primaria en regresión (árbol de decisión):

- 1. ¿Hay **outliers** que NO deben dominar? → **MAE** o **MedAE** (errores absolutos, robustos).
- 2. ¿Los errores grandes son **especialmente** malos (deben penalizar más)? → **RMSE / MSE**.
- 3. ¿Necesitas un error **relativo** (%) y no hay valores cerca de 0? → **MAPE** (con cuidado).
- 4. ¿Quieres comunicar **"% de varianza explicada"**? → **R²** (ajustado si comparas modelos con distinto nº de features).
- 5. ¿El target crece de forma **multiplicativa/exponencial**? → **RMSLE**.

 **INTUICIÓN MATEMÁTICA — MAE vs RMSE (la diferencia que más se pregunta).** Ambas miden error en las unidades del target, pero la RMSE **eleva al cuadrado** antes de promediar, así que un error de 10 pesa 100 mientras que diez errores de 1 pesan 10: la RMSE **castiga desproporcionadamente los errores grandes** (y, por tanto, los outliers). MAE trata todos por igual. Regla práctica: si te duelen mucho los fallos gordos → RMSE; si quieres robustez a casos extremos → MAE. Además, **RMSE ≥ MAE siempre**; cuanto más se separan, más dispersos (heterogéneos) son tus errores.

 **INTUICIÓN MATEMÁTICA — leer el R².** $R^2 = 1$ → predicción perfecta. $R^2 = 0$ → tu modelo iguala a "predecir siempre la media" (no aporta nada). $R^2 < 0$ → tu modelo es **peor** que predecir la media (sí, puede pasar, sobre todo en test). No es un "porcentaje de acierto": es la **fracción de la varianza** del target que el modelo explica. Un R^2 de 0.85 significa que el modelo da cuenta del 85 % de la variabilidad; el 15 % restante es ruido o señal no capturada.

 **ERROR TÍPICO (entrevista) — pedir "el recall de mi regresión".** *Error:* aplicar Recall/F1/Accuracy a un problema de regresión. *Consecuencia:* no están definidos (no hay clases). La respuesta correcta: en regresión se usan **MAE/RMSE/R²**, etc. Si de verdad necesitas clases, primero **discretiza** el target (binning) y entonces vuelves a un problema de clasificación — pero entonces el problema cambia.

 **ERROR TÍPICO — MAPE con valores cercanos a cero.** *Error:* usar MAPE cuando el target puede ser 0 o casi. *Consecuencia:* la división por ~0 dispara la métrica al infinito y la vuelve inútil/engañosa. Alternativas: MAE, sMAPE o RMSLE.

20.3 · Tabla puente: ¿qué métrica para qué problema?

Tipo de problema (estático)	Target	Métricas válidas (primarias)	Métricas que NO aplican	Métrica anti-engaño
Clasificación binaria equilibrada	categoría 0/1	Accuracy, F1, ROC-AUC	MAE, RMSE, R²	—
Clasificación binaria desbalanceada	categoría 0/1 (rara)	F1, MCC, PR-AUC , Recall@precisión	Accuracy (engaña), MAE/RMSE/R²	MCC / PR-AUC
Clasificación multiclase	3+ categorías	F1 macro/weighted, MCC multiclase, Kappa	MAE/RMSE/R², "recall" sin promediar	F1 macro (trata clases por igual)
Regresión	número continuo	MAE, RMSE, R² , MAPE, MedAE	Accuracy, Precision, Recall, F1, MCC, AUC	R² ajustado (vs añadir features)
Ranking / priorización	orden, no clase	ROC-AUC, PR-AUC, NDCG, MAP	Accuracy "dura", MAE/RMSE	PR-AUC con desbalance

 **QUÉ DEBES RECORDAR (métricas)**

- 1. El **tipo de target manda**: categoría → métricas de confusión/probabilidad; número → métricas de distancia.
- 2. **Recall, F1, Accuracy, MCC, AUC son SOLO de clasificación**. **MAE, RMSE, R², MAPE son SOLO de regresión**. No se cruzan porque miden cosas distintas (acierto de etiqueta vs distancia numérica).
- 3. En clasificación **desbalanceada**: F1/MCC/PR-AUC sí, Accuracy no. En **regresión**: MAE/MedAE si hay outliers, RMSE si los errores grandes deben doler, R² para comunicar varianza explicada.
- 4. **R² puede ser negativo** (peor que la media); **MAPE explota** cerca de 0; **RMSE ≥ MAE** siempre.
- 5. Las **series de tiempo** son otra liga: mismas métricas de error + específicas (MASE, sMAPE) + validación temporal (nunca barajar).
- 6. Elige **una métrica primaria** alineada con el negocio y reporta varias de apoyo; no optimices ninguna que tenga un **óptimo degenerado trivial** (recall puro).

21 · Tabla maestra de referencia rápida (chuleta profesional)

Esta es la **chuleta** para repasar de un vistazo todo el flujo de un proyecto de clasificación (y parte del de regresión). Está dividida por bloques temáticos para que la encuentres rápido. Columnas: **Concepto** · **Qué significa** · **Cuándo usarlo** · **Cuándo evitarlo** · **Riesgo principal** · **Error típico** · **Recomendación práctica**.

21.1 · Datos, auditoría e ingeniería de variables

Concepto	Qué significa	Cuándo usarlo	Cuándo evitarlo	Riesgo principal	Error típico	Recomendación
Cardinalidad / ID	Nº de valores únicos de una variable	Auditar toda variable	—	ID como feature = memorización	Dejar <code>customerid</code> en X	Elimina IDs; vigila alta cardinalidad
Mutual Information	Info compartida con el target (no lineal)	Ranear señal de variables	Como único criterio para eliminar	Es univariada (ignora interacciones)	Tirar variables con $MI \approx 0$	Úsala para priorizar, no para podar sola
Multicolinealidad / VIF	Variables predictoras correlacionadas entre sí	Antes de interpretar lineales	Si solo usas árboles para predecir	Coeficientes inestables	Interpretar coef. con colinealidad alta	$VIF > 5-10 \rightarrow$ reduce/regulariza/PCA
Techo de Bayes	Error mínimo irreducible del problema	Fijar expectativas	—	Perseguir métricas imposibles	Buscar $F1 \approx 0.95$ donde el techo es 0.64	Acéptalo; solo baja con features nuevas
Desbalance de clases	Una clase mucho más rara que otra	Clasificación con clase rara	Clases equilibradas	El modelo ignora la minoritaria	Usar Accuracy como métrica	Pesos de clase + umbral + F1/MCC/PR-AUC
Parsimonia (Occam)	Preferir lo simple a igual rendimiento	Selección final, ingeniería	Si lo simple rinde peor de verdad	Complejidad inútil = más varianza	Añadir features "por si acaso"	Cada feature debe mejorar la CV

21.2 · Validación, leakage y estadística

Concepto	Qué significa	Cuándo usarlo	Cuándo evitarlo	Riesgo principal	Error típico	Recomendación
Data leakage	Información prohibida que se filtra	Vigilarlo siempre	—	Métricas infladas que se caen en prod	Escalar antes del split	Todo en pipelines; test ciego
Train/Test aislado	Muestra final intacta	Todo proyecto	—	Quemar el test al mirarlo	Elegir features/umbral en test	Tócalo una vez, al final
Estratificación	Mantener proporción de clases	Clasificación desbalanceada	Series de tiempo	Folds con pocos positivos	No estratificar con 90/10	<code>stratify=y</code> en split y folds
K-Fold / SKF / RSKF	Validación cruzada (repetida/estratíf.)	Datasets peq./medianos	Datos temporales/agrupados	Hold-out único ruidoso	Comparar en 1 partición	RSKF para comparar ($SEM \approx s/\sqrt{k}$)
SEM / IC 95 %	Precisión de la media estimada	Reportar incertidumbre	—	Confundir ruido con señal	Dar una cifra sin intervalo	Reporta media $\pm$ IC
Wilcoxon pareado	Test no paramétrico sobre folds	Comparar 2 modelos en mismos folds	Muestras no pareadas	Declarar ganador por ruido	"Gana por 0.001 de F1"	$p > 0.05$ = empate $\rightarrow$ elige simple
Comparaciones múltiples	Inflar falsos positivos al testear mucho	Muchas comparaciones	1 sola comparación	"Hallazgos" que son azar	No corregir con 8 tests	Bonferroni/Holm si decides sobre uno
Nested CV	Tuning interno + evaluación externa	Tunear + reportar generalización	Sin tuning / sin presupuesto	Score de tuning optimista	Reportar CV de tuning como real	Úsala para la cifra final

21.3 · Preprocesamiento y modelos

Concepto	Qué significa	Cuándo usarlo	Cuándo evitarlo	Riesgo principal	Error típico	Recomendación
Escalado	Poner numéricas en rango comparable	Distancia/kernel/lineal/red	Árboles/boosting	Variable de gran rango domina	Escalar antes del split (leakage)	Standard (o Robust con outliers), en pipeline
One-Hot Encoding	Una columna 0/1 por categoría	Nominal, baja cardinalidad	Alta cardinalidad; ordinal real	Explosión de columnas si card. alta	Aplicarlo a la target	Por defecto para nominales; <code>drop='if_binary'</code>
Ordinal Encoding	Enteros por categoría	Orden real ($S < M < L$)	Nominales sin orden	Orden inventado engaña	Ordinal a método de pago	Solo con orden verdadero
Target Encoding	Categoría $\rightarrow$ media del target	Alta cardinalidad	Baja cardinalidad	Leakage si media global	Media con todo el dataset	CV interna, dentro del pipeline

Concepto	Qué significa	Cuándo usarlo	Cuándo evitarlo	Riesgo principal	Error típico	Recomendación
Pipeline / ColumnTransformer	Preproceso+modelo en un objeto	Siempre que haya preproceso	—	Leakage y skew sin él	Preproceso fuera de la CV	Encapsula todo; serializa el pipeline
Class weight / scale_pos_weight	Penalizar más la clase rara	Desbalance	Desbalance leve	Descalibra probabilidades	Olvidar recalibrar	Primera opción vs SMOTE; recalibra si usas prob.
Bagging (RF/ExtraTrees)	Promediar árboles (↓varianza)	Robustez, baseline fuerte	Si lo lineal ya basta	Puede memorizar (gap alto)	Confiar en feature_importances_	Mira el gap; usa permutation importance
Boosting (XGB/LGBM/Cat/HistGB)	Árboles secuenciales (↓sesgo)	Patrón no lineal explotable	Patrón casi lineal	Sobreajuste sin regular.	No usar early stopping	Regulariza + early stopping; vigila gap
Regularización (C, L1/L2)	Penalizar complejidad	Lineales, muchas features	—	Demasiada → underfit	No escalar antes de regular.	Tunea C por CV; L1 si quieres selección

21.4 · Optimización, interpretabilidad y despliegue

Concepto	Qué significa	Cuándo usarlo	Cuándo evitarlo	Riesgo principal	Error típico	Recomendación
Optuna / TPE	Optimización bayesiana de hiperparám.	Espacio grande/caro	Espacio trivial	Sobreajuste de tuning	Tunear mirando el test	CV siempre; confirma con nested CV
Pruning	Abortar pruebas malas pronto	Tuning con CV por folds	Pocos trials	Matar configs lentas	Pruning sin warmup	n_warmup_steps prudente
Early stopping	Parar de añadir árboles al estancarse	Boosters	Modelos sin iteraciones	Leakage en eval_set	eval_set mal transformado	Transforma con el fold-train
Curva de aprendizaje	F1 train/CV vs tamaño muestra	Diagnóstico de datos	—	Malinterpretar sesgo/varianza	Trazarla en test	Out-of-fold; decide datos vs modelo
Calibración	Probabilidades = tasas reales	Si usas la probabilidad	Si solo importa la clase	Prob. mentirosas (pesos de clase)	Confiar en prob. sin verificar	CalibratedClassifierCV si hace falta
Permutation importance	Caída de métrica al barajar variable	Importancia global fiable	Features muy correlacionadas	Combos imposibles distorsionan	Usar feature_importances_	Model-agnostic, sobre la métrica
SHAP	Aporte por feature en cada caso	Explicar casos individuales	Causalidad	Confundir con causa	"Quita fibra y retienes"	Explica el modelo, no el mundo
Umbral de decisión	Corte prob. → clase	Desbalance/pesos de clase	—	0.5 subóptimo	Optimizar umbral en test	Optimiza out-of-fold (o por coste)
Training-serving skew	Preproceso prod ≠ train	Todo despliegue	—	Degradación silenciosa	Recodificar a mano en prod	Empaqueta el pipeline completo
Data drift	La población cambia en producción	Monitorización continua	—	Modelo obsoleto en silencio	No monitorizar	Vigila distribución; reentrena

21.5 · Métricas — qué usar según el problema

Concepto	Qué significa	Cuándo usarlo	Cuándo evitarlo	Riesgo principal	Error típico	Recomendación
Accuracy	% aciertos totales	Clasif. equilibrada	Desbalance	La infla la mayoritaria	"99 % accuracy" con 99/1	Solo con clases equilibradas
Precision	Aciertos entre los predichos +	FP caros	FN es lo grave	Ignora positivos perdidos	Optimizarla sola	Acompañala de recall
Recall	Positivos reales detectados	FN caros	Sola (óptimo degenerado)	Predecir todo +	refit='recall'	Nunca en solitario; usa F1/F-beta
F1	Media armónica P y R	Desbalance, ambos errores	Costes muy asimétricos	No mide calibración	Creer F1 alto = prob. buenas	Métrica primaria con desbalance
MCC	Correlación real-predicho (4 casillas)	Desbalance (lo más honesto)	Si necesitas algo "de negocio"	Menos intuitiva	Ignorarla por poco famosa	Repórtala junto a F1
ROC-AUC	Calidad del ranking	Comparar ordenación	Desbalance fuerte	Optimista con muchos negativos	Reportarla sola con 99/1	Con desbalance, prefiere PR-AUC

Concepto	Qué significa	Cuándo usarlo	Cuándo evitarlo	Riesgo principal	Error típico	Recomendación
PR-AUC	Área Precision-Recall	Desbalance, clase rara	Clases equilibradas	Depende de la tasa base	Comparar entre tasas base	Métrica de cabecera en desbalance
MAE (regresión)	Error absoluto medio	Robustez a outliers	Si grandes errores deben doler	Trata todo igual	Usarla con "recall" mental	Error en unidades reales
RMSE (regresión)	Raíz del error cuadrático	Penalizar errores grandes	Outliers que no deben dominar	Sensible a extremos	Comparar entre escalas	RMSE≥MAE; úsala si los fallos gordos importan
R² (regresión)	Varianza explicada	Comunicar ajuste	Comparar datasets distintos	Puede ser negativo	"R² alto = buen modelo"	R² ajustado al comparar nº features
MAPE (regresión)	Error porcentual medio	Error relativo (%)	Valores cerca de 0	Explota cerca de 0	Usarla con ceros	Evítala si el target pasa por 0

Regla mnemónica final.

- **Clasificación** = "¿acerté la etiqueta?" → Accuracy, Precision, Recall, F1, MCC, ROC-AUC, PR-AUC.
- **Regresión** = "¿cuánto me desvié del número?" → MAE, RMSE, R², MAPE, MedAE.
- **Nunca cruces las listas.** Si alguien te pide "el recall de tu regresión" o "el RMSE de tu clasificación", hay un malentendido sobre el tipo de problema.

Epílogo · Cómo convertir esto en intuición permanente

Has llegado al final. Si tuvieras que quedarte con **diez ideas** que distinguen a un buen data scientist de uno excelente, serían estas:

1. **El objetivo es el dato que aún no existe.** Optimiza generalización, no la métrica que ves hoy.
2. **El leakage es el enemigo silencioso.** Suele ser de proceso (escalar antes de separar, elegir mirando el test), no de columnas. Los pipelines y el test aislado lo matan por construcción.
3. **Una diferencia no es una mejora hasta que sobrevive a la estadística.** IC, Wilcoxon, nested CV: ruido ≠ señal.
4. **A igualdad de rendimiento, gana lo simple.** Menos varianza, más robustez, más interpretabilidad, menos cosas que se rompan.
5. **La métrica la dicta el target.** Clasificación → confusión/probabilidad; regresión → distancia. Y nunca optimices una métrica con óptimo degenerado (recall puro).
6. **Conoce tu techo (de Bayes).** Persigue lo alcanzable; un número demasiado bueno delata leakage, no genialidad.
7. **Escala lo que mide distancias; no escales árboles.** Codifica nominales con One-Hot; reserva target encoding (con CV) para alta cardinalidad.
8. **Interpreta con varios métodos y desconfía de la causalidad.** SHAP/permutation/coeficientes explican el modelo, no el mundo.
9. **Despliega el pipeline entero y vigila el drift.** El modelo de hoy es el modelo obsoleto de mañana si nadie lo mira.
10. **Documenta el porqué, no solo el qué.** Una decisión defendible ante una auditoría vale más que una métrica vistosa.

El hilo conductor de todo el documento: cada técnica existe para **proteger de un error concreto**. Estratificar protege de folds degenerados; los pipelines, del leakage; el Wilcoxon, de confundir ruido con señal; la nested CV, del optimismo del tuning; Occam, de la varianza inútil. Si entiendes *de qué te protege* cada herramienta, sabrás siempre cuándo usarla y cuándo no, incluso en problemas que este documento no cubre.

Documento generado como material de estudio de nivel profesional. Reproducible end-to-end ejecutando [mejores_practicas.ipynb](#) con la semilla fijada. Las cifras (F1, MCC, p-valores, gaps, IC) provienen de esa ejecución y son auditables. Lectura complementaria: [informe_auditoria.md](#) (diagnóstico del notebook original).